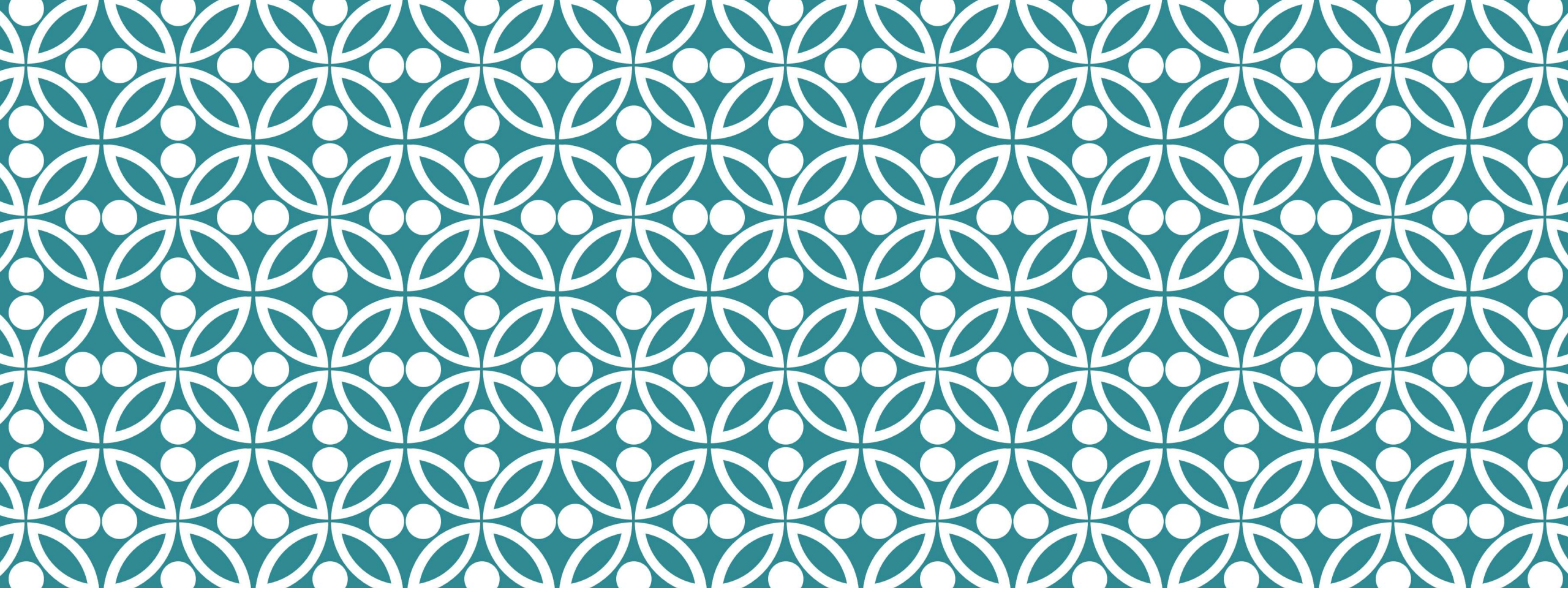
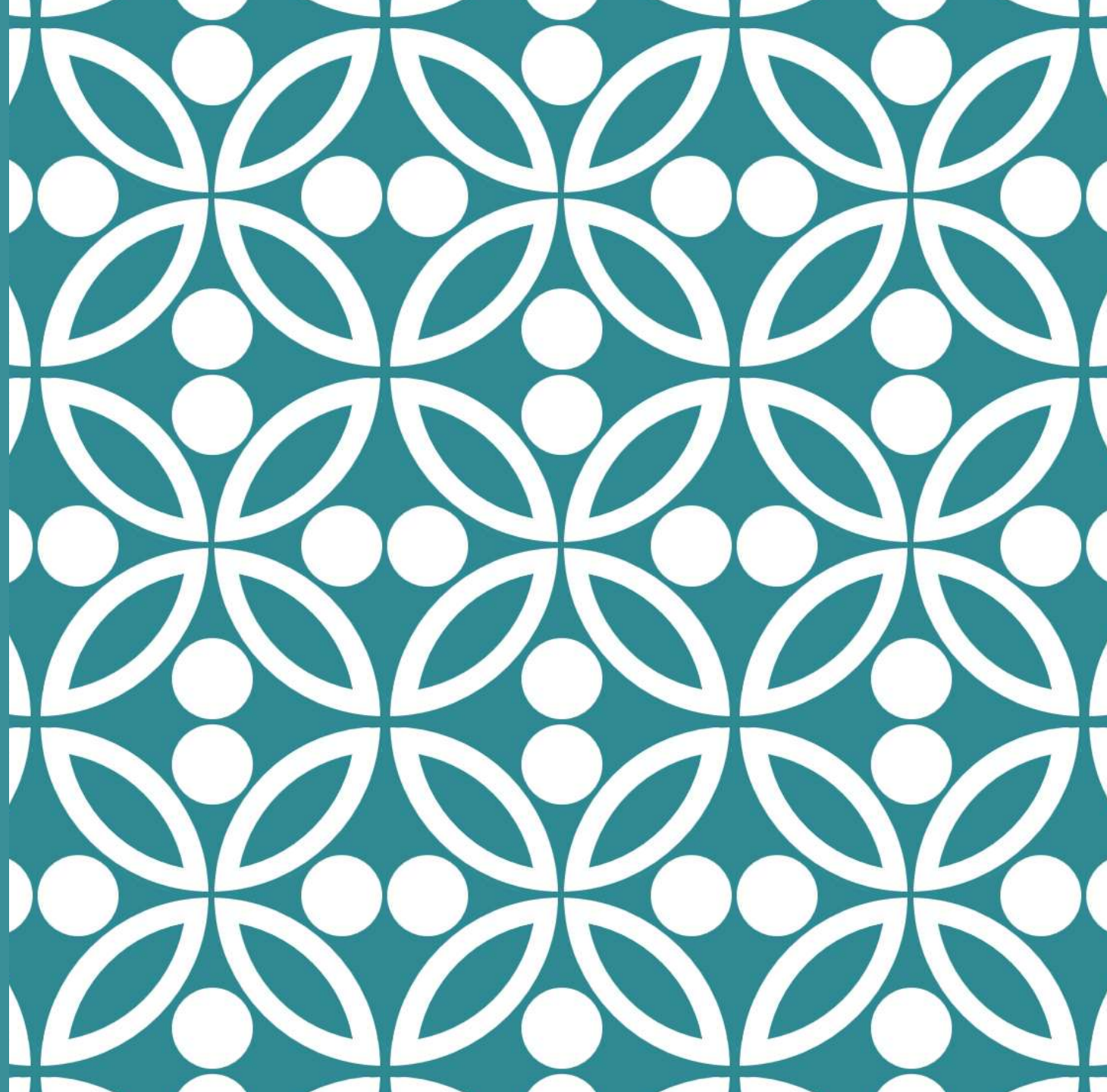


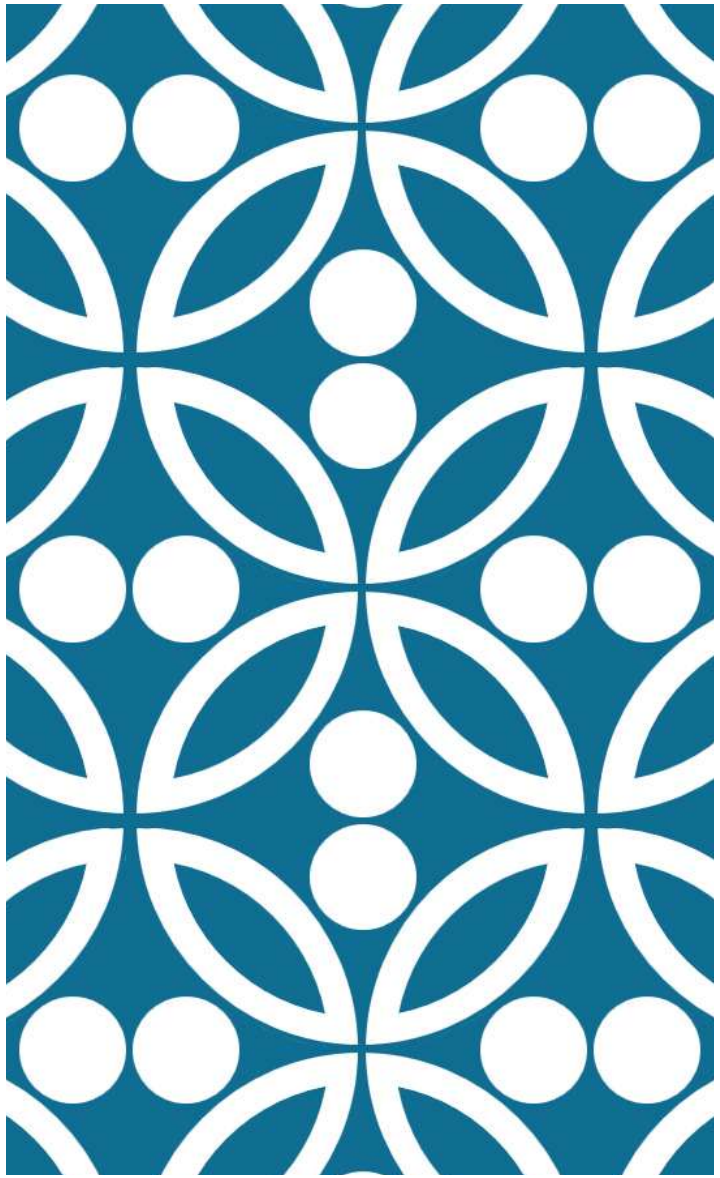
SESSION 2: R-MARKDOWN

DR. SOFIA GIL-CLAVEL

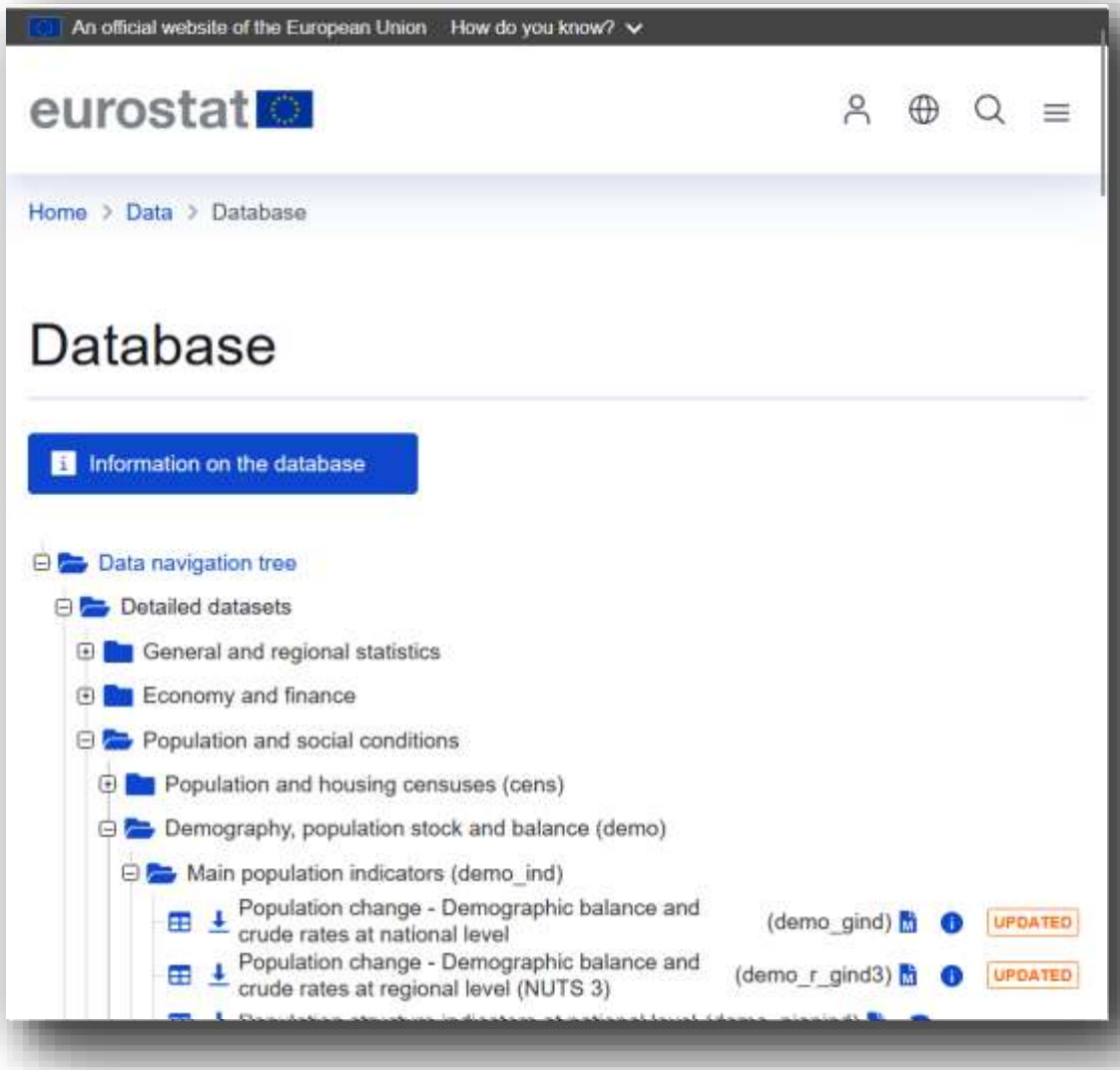
- Introduction: Creating and rendering documents
- Adding and formatting text
- Modifying the settings
- Building on axioms
- Tracking students' logic

1. INTRODUCTION: CREATING AND RENDERING DOCUMENTS



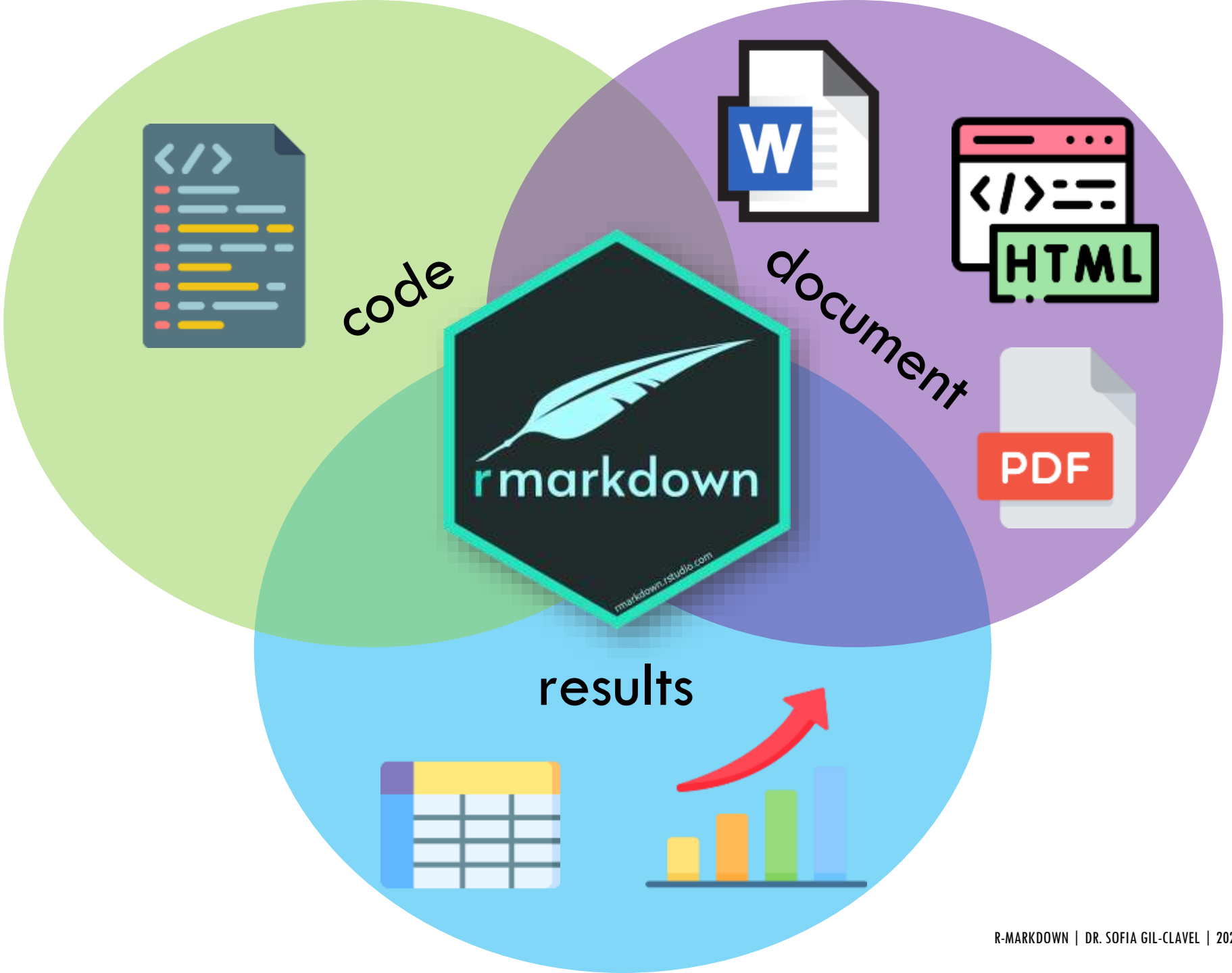


INTRODUCTION



HAS IT HAPPENED TO YOU THAT...?

What if there is a R package that helps you have everything in the same place?


code
document
results
rmarkdown
HTML
PDF

Source: https://rmarkdown.rstudio.com/

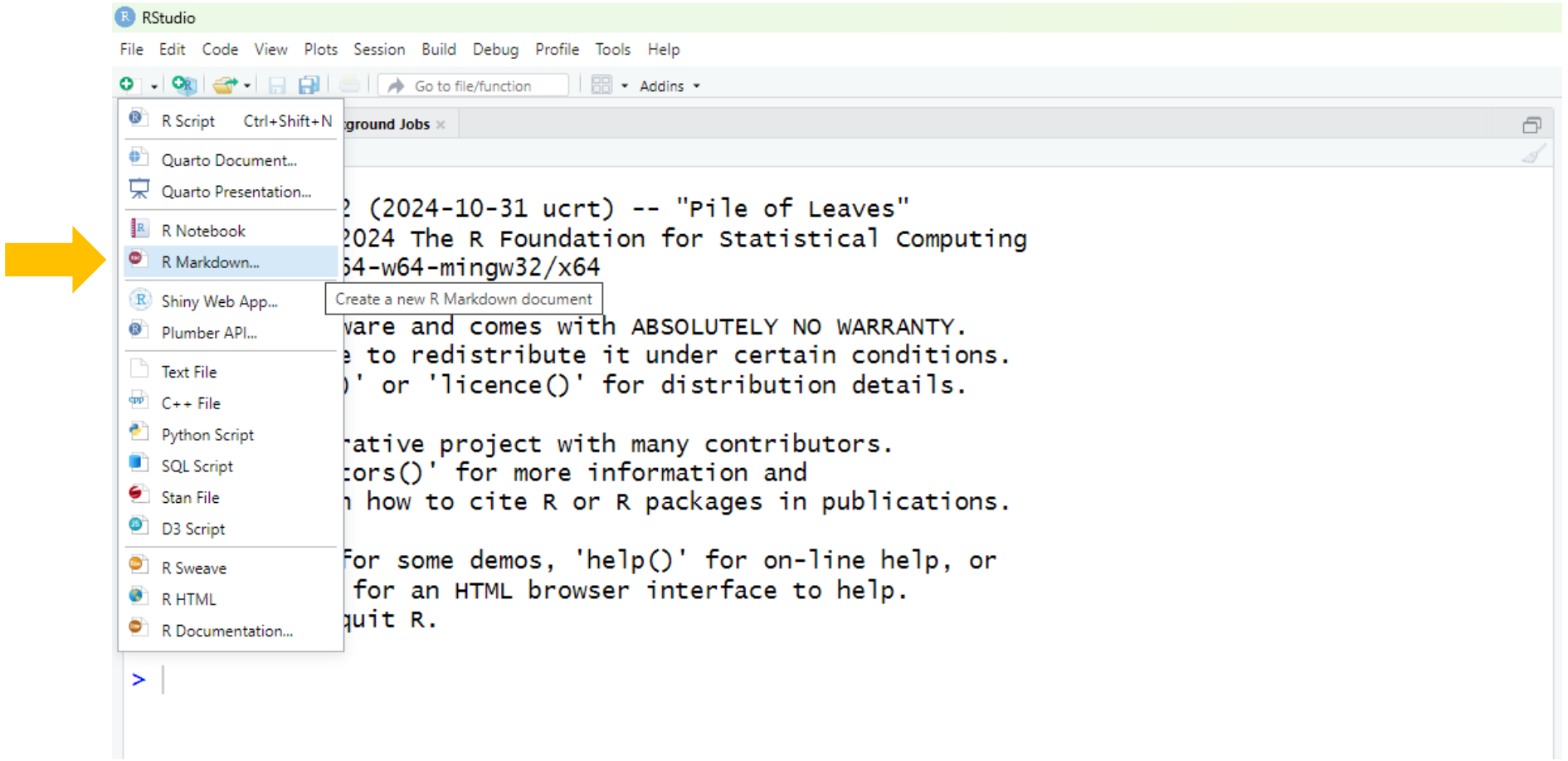
BUT FIRST, WE NEED TO INSTALL LATEX

Just write and run the following command:

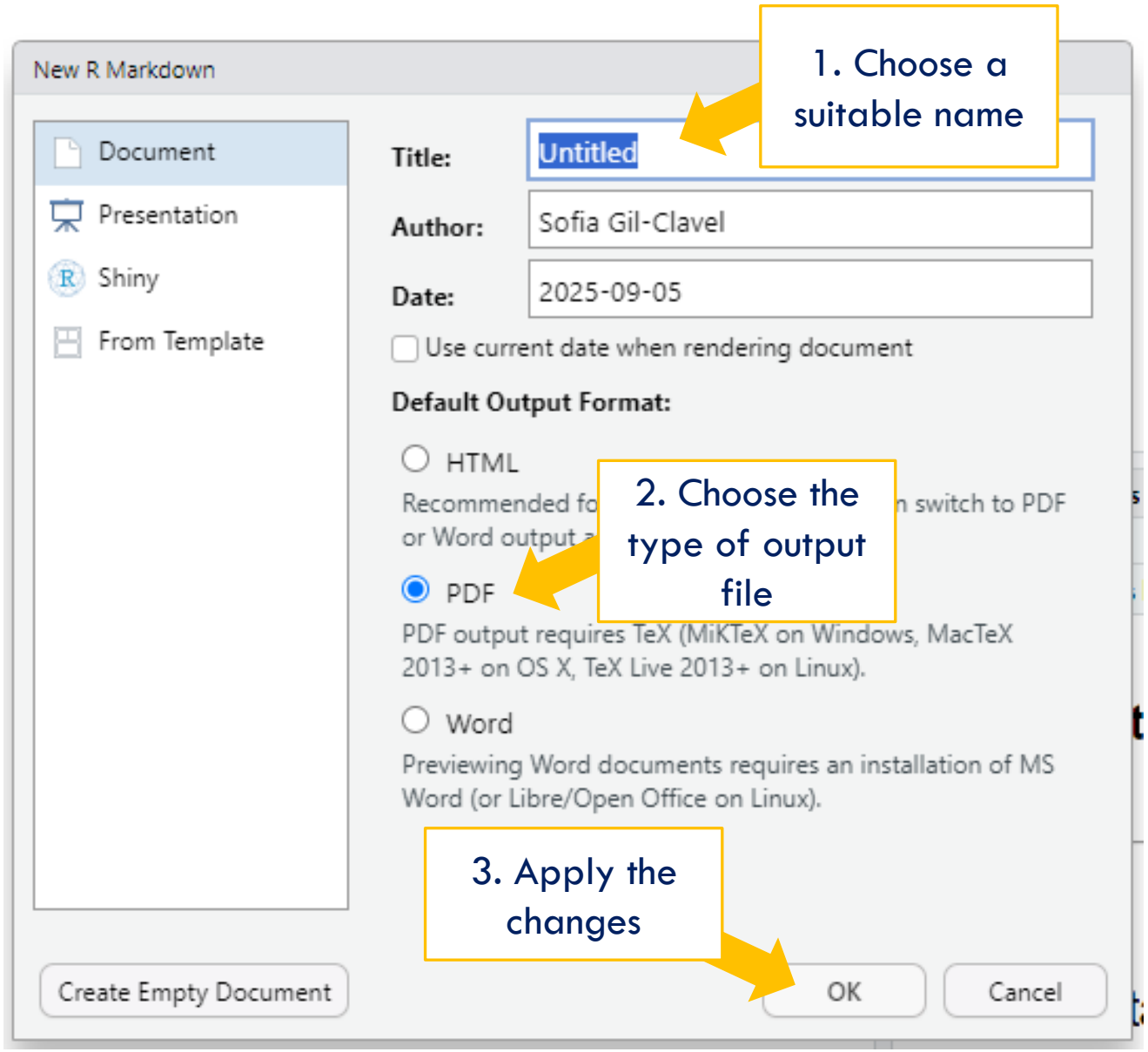
```
tinytex::install_tinytex()
```

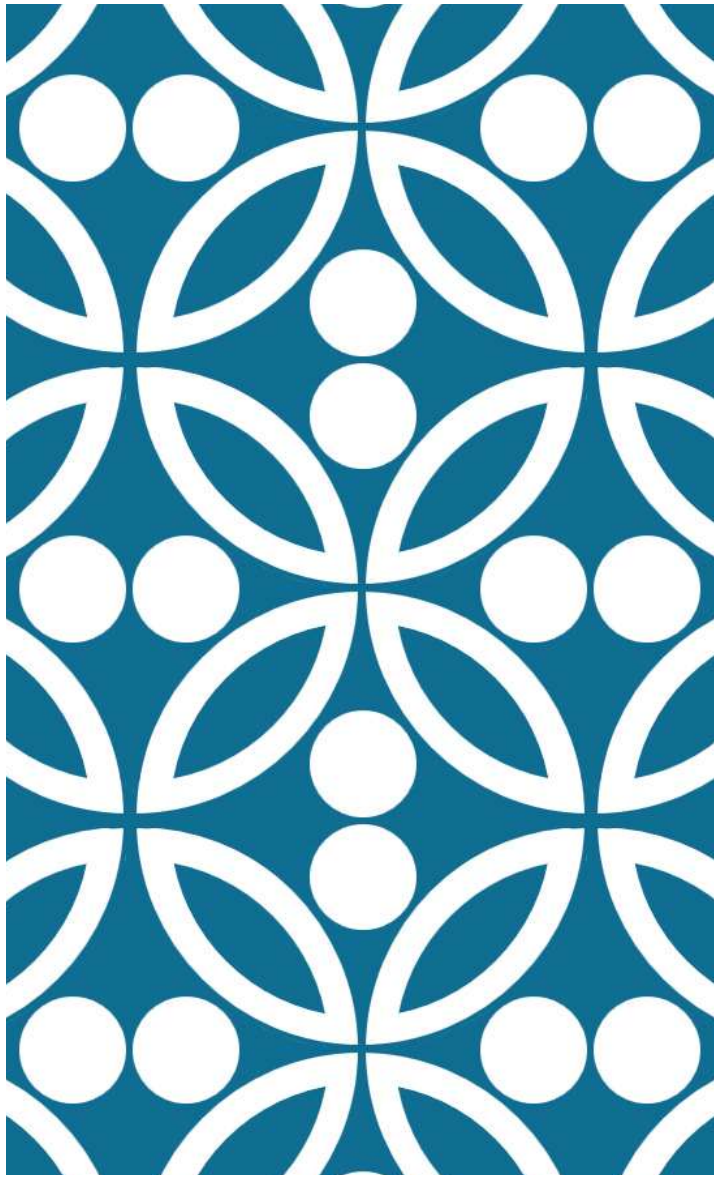
It will download and install a small version of a software called LaTeX. This software is kind of like Word, but it is mostly used by mathematicians and programmers because it allows writing math and pseudo code in an easy way. It is also for free.

OPEN A R-MARKDOWN SCRIPT



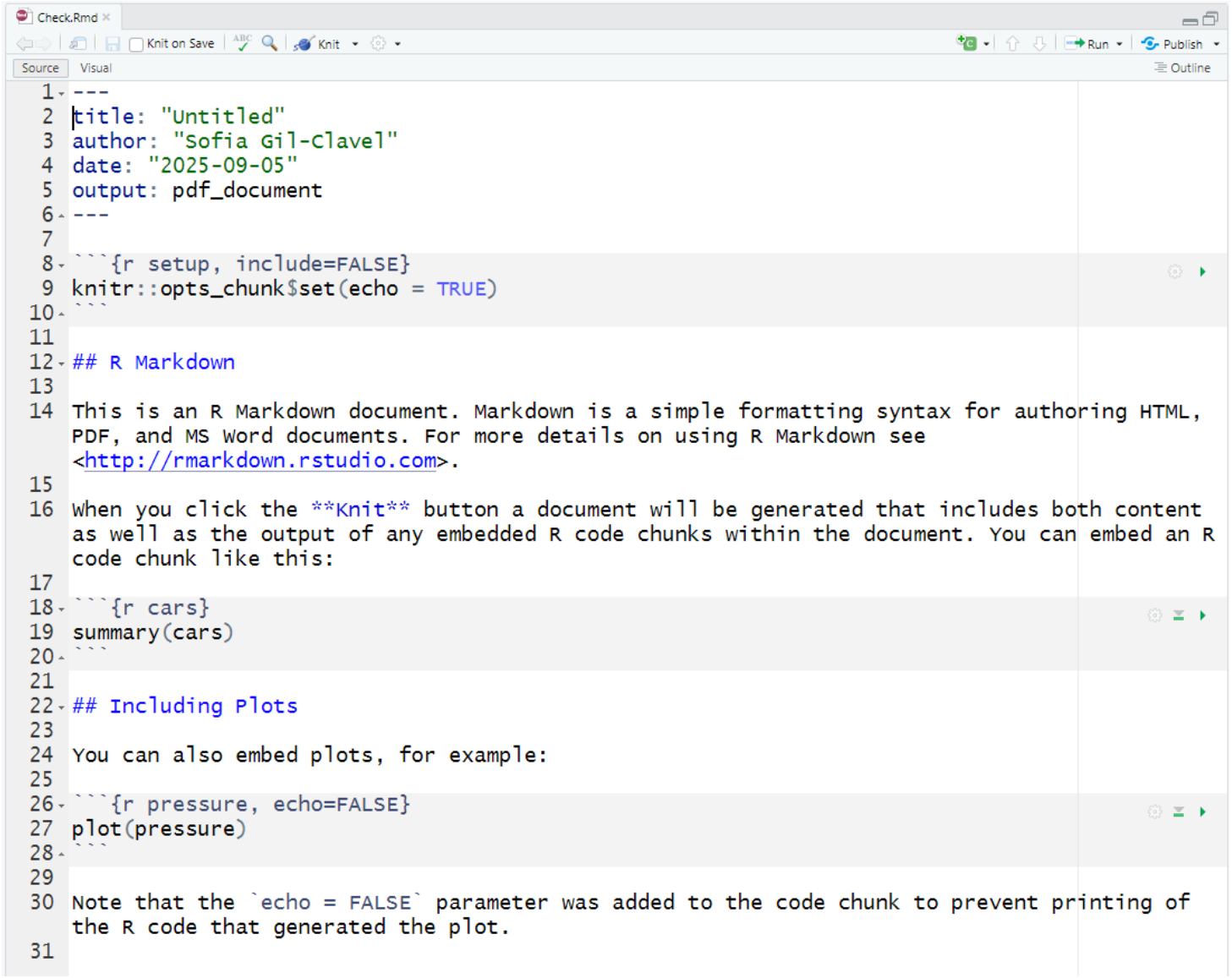
SET UP THE FILE:



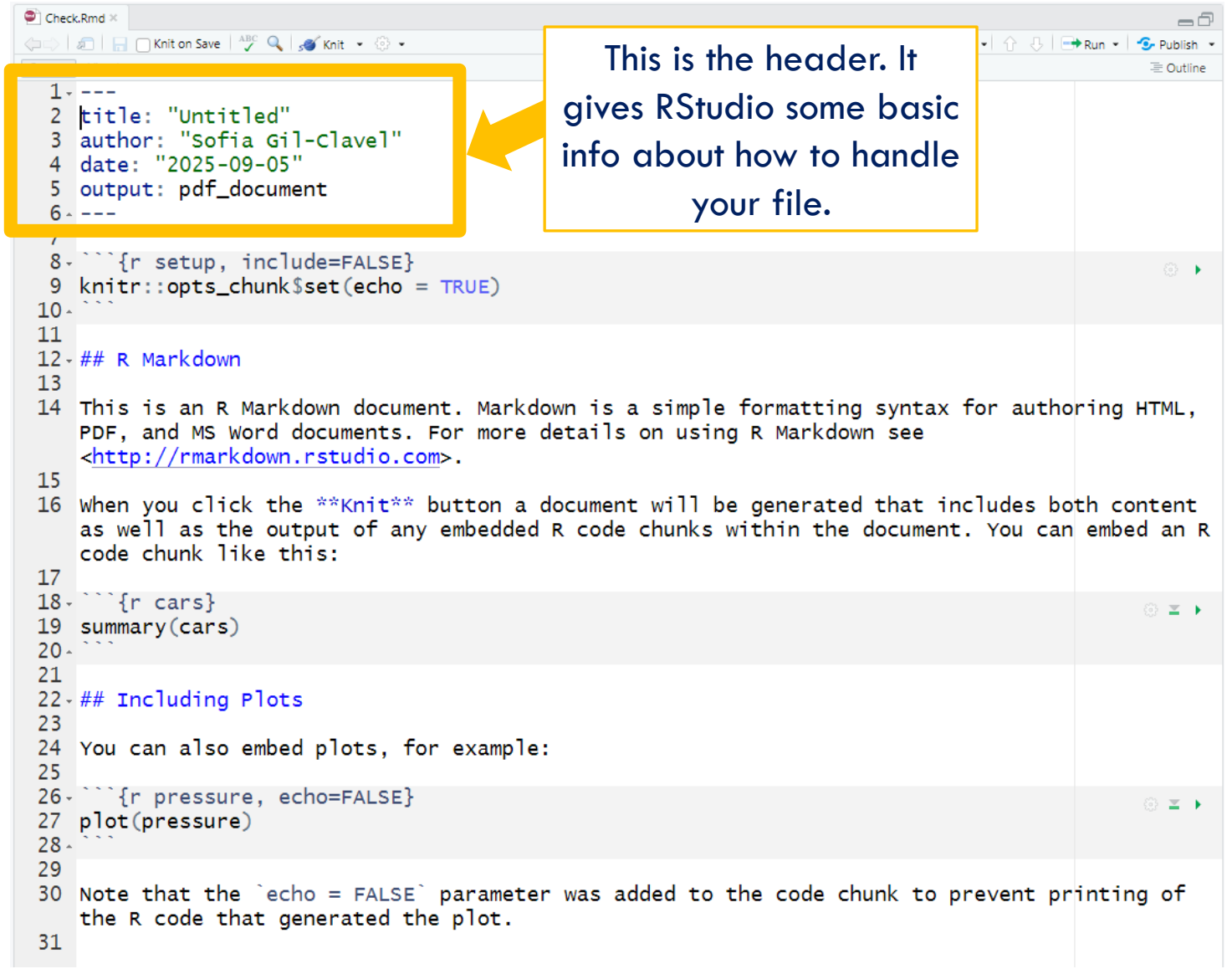


THE R-MARKDOWN ELEMENTS

THIS OPENS A SCRIPT THAT LOOKS LIKE:

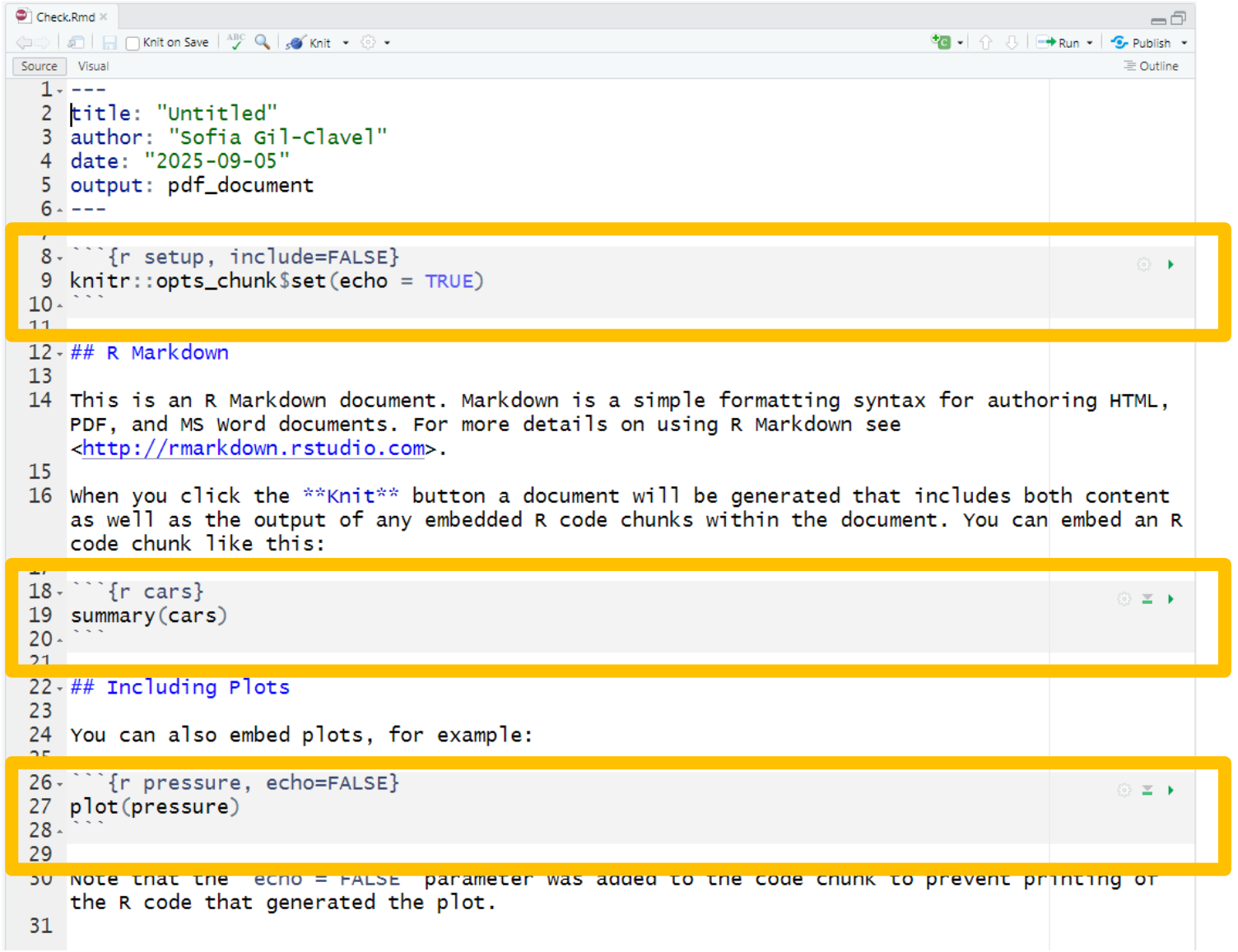


THIS OPENS A SCRIPT THAT LOOKS LIKE:



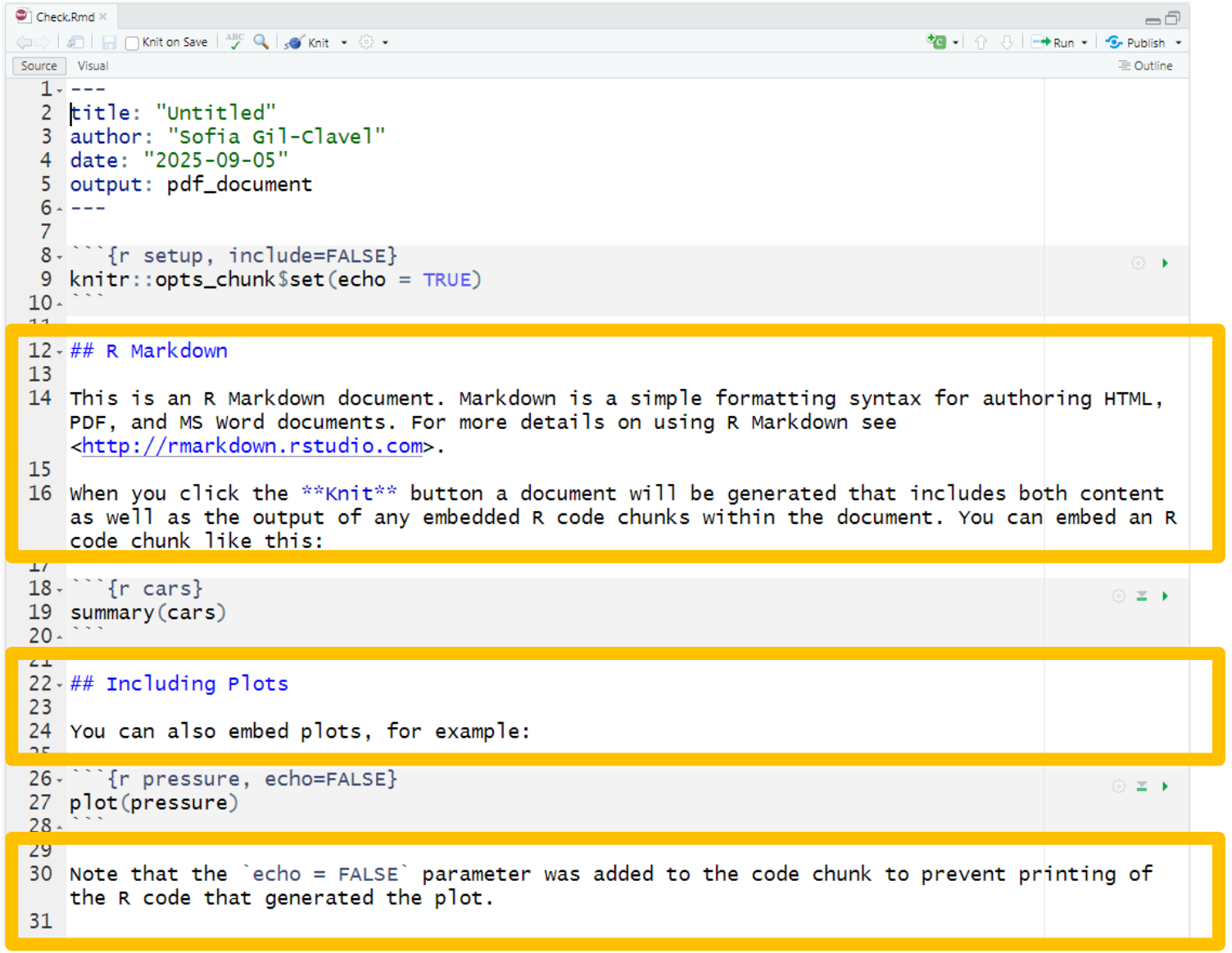
THIS OPENS A SCRIPT THAT LOOKS LIKE:

These are called "Chunks". In them you can write code.



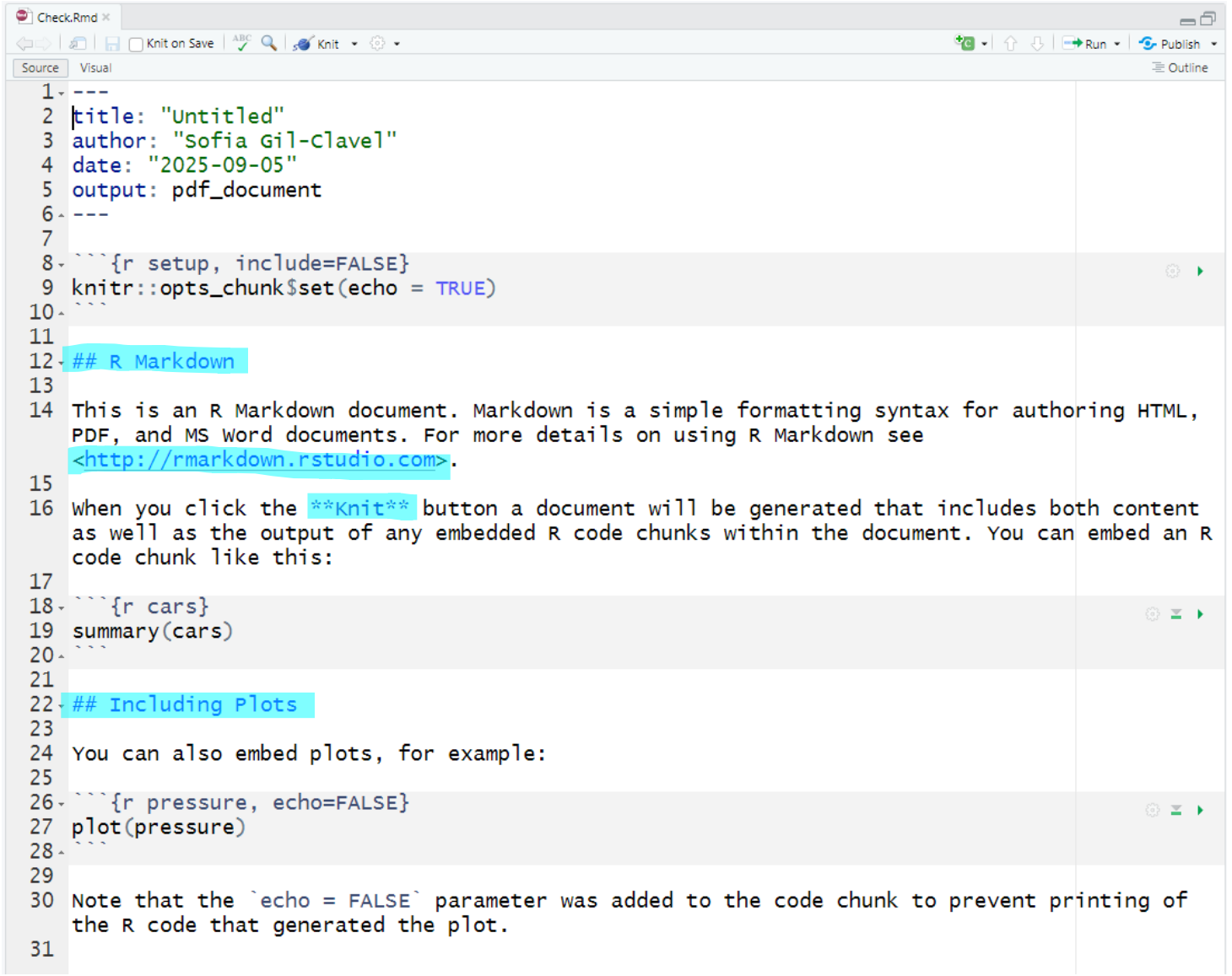
THIS OPENS A SCRIPT THAT LOOKS LIKE:

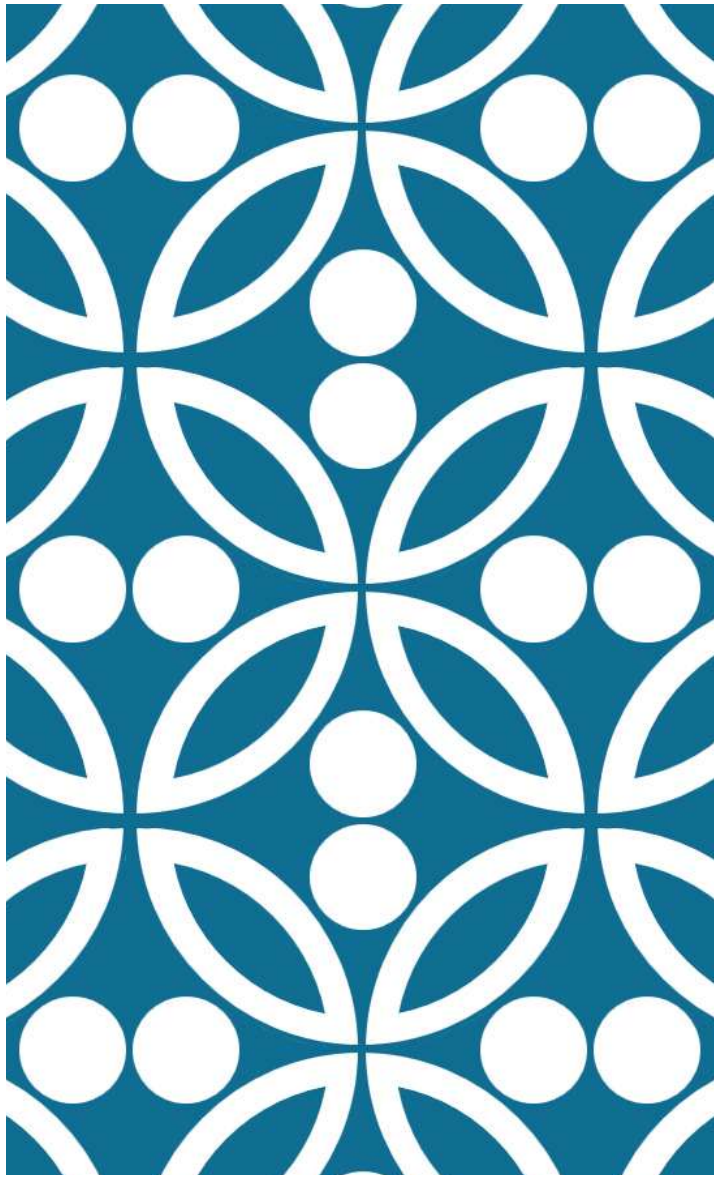
You can write normal text outside the chunks.



THIS OPENS A SCRIPT THAT LOOKS LIKE:

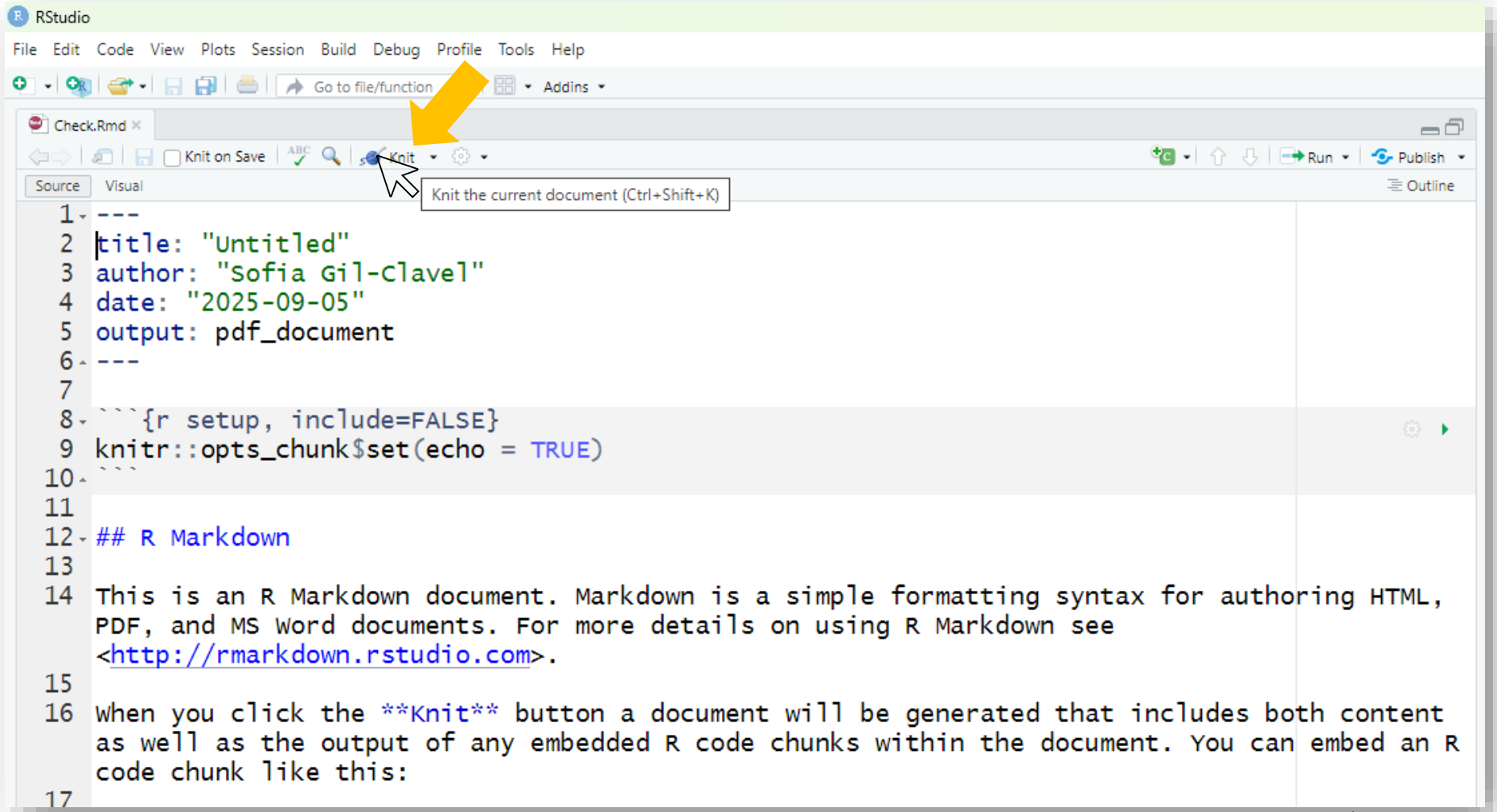
This is special syntax that changes how the text looks in the final document.





FROM R-MARKDOWN TO PDF, WORD, OR HTML.

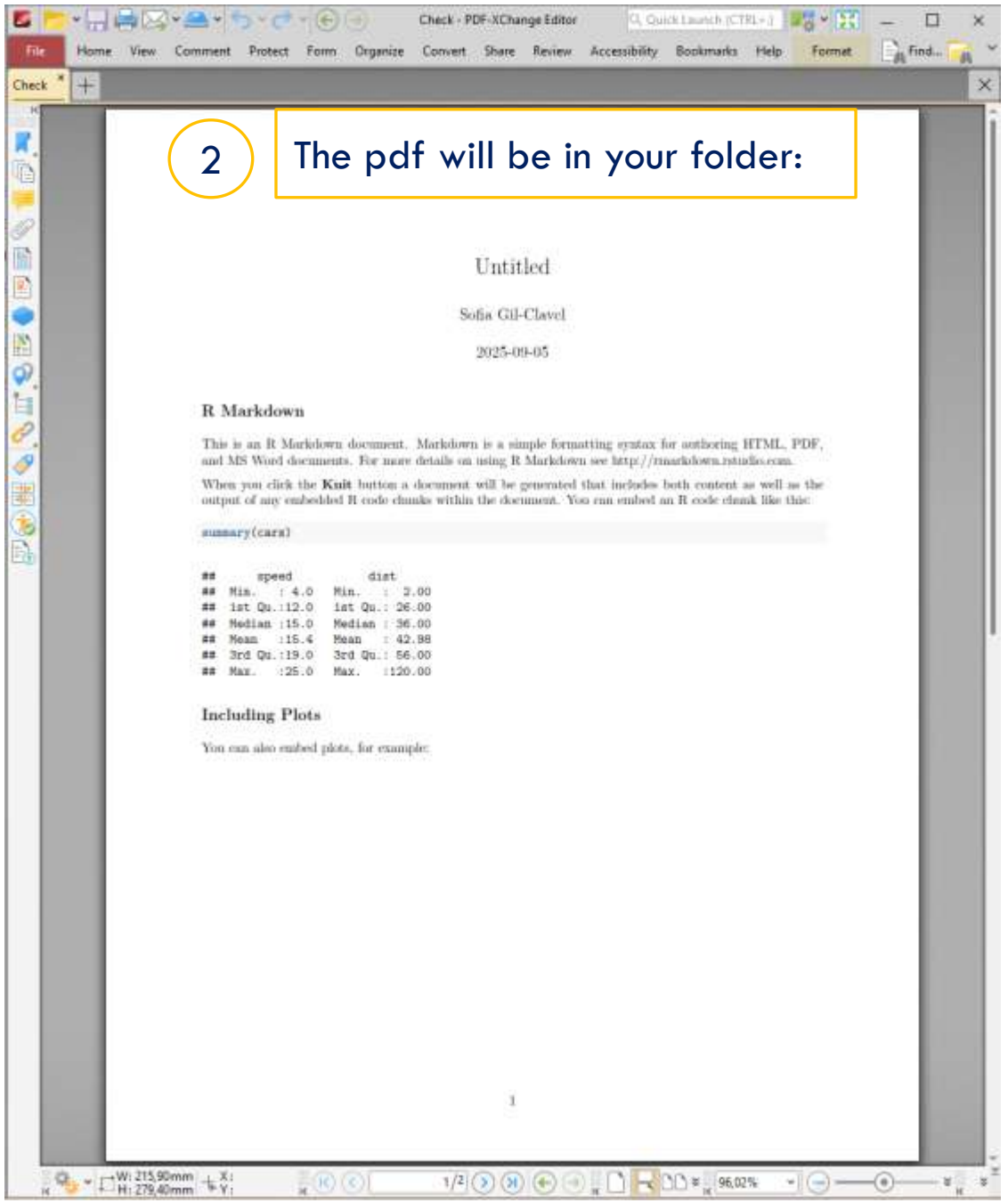
LET'S RENDER YOUR R-MARKDOWN FILE

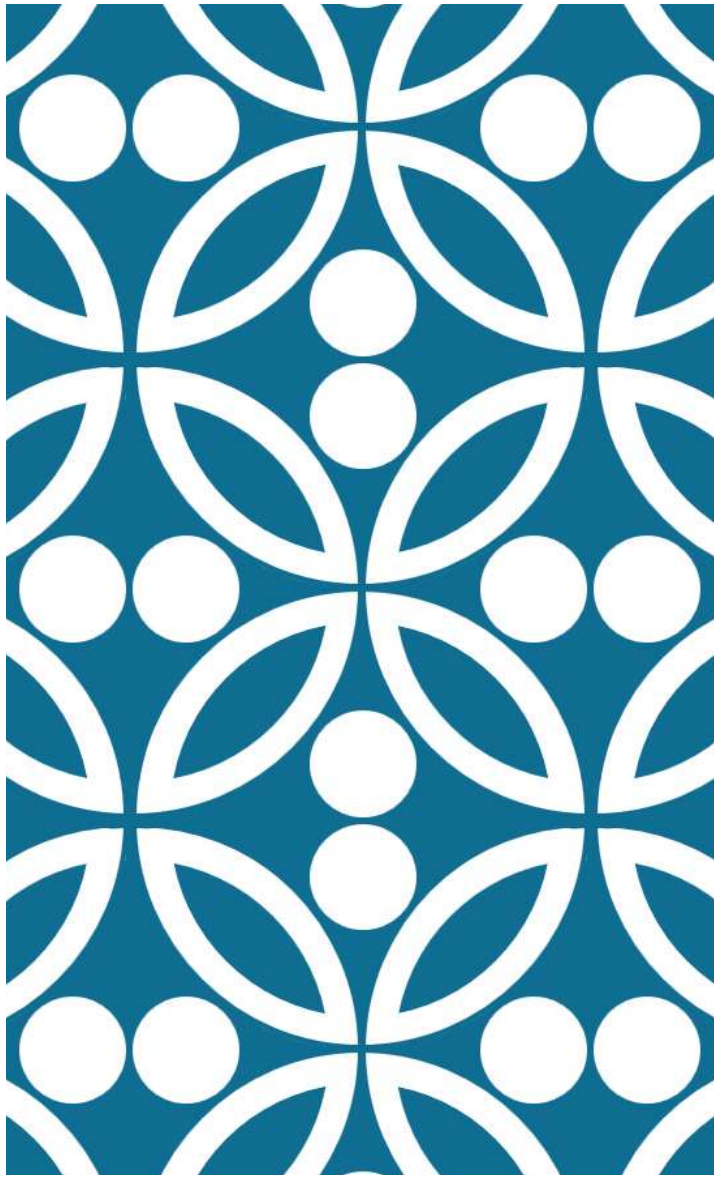


IF EVERYTHING GOES WELL THEN…

1 The panel "Render" will display something similar to:

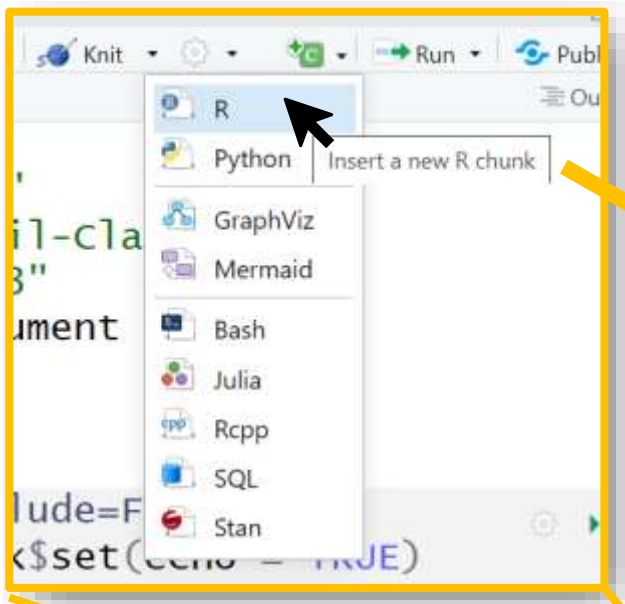
2 The pdf will be in your folder:

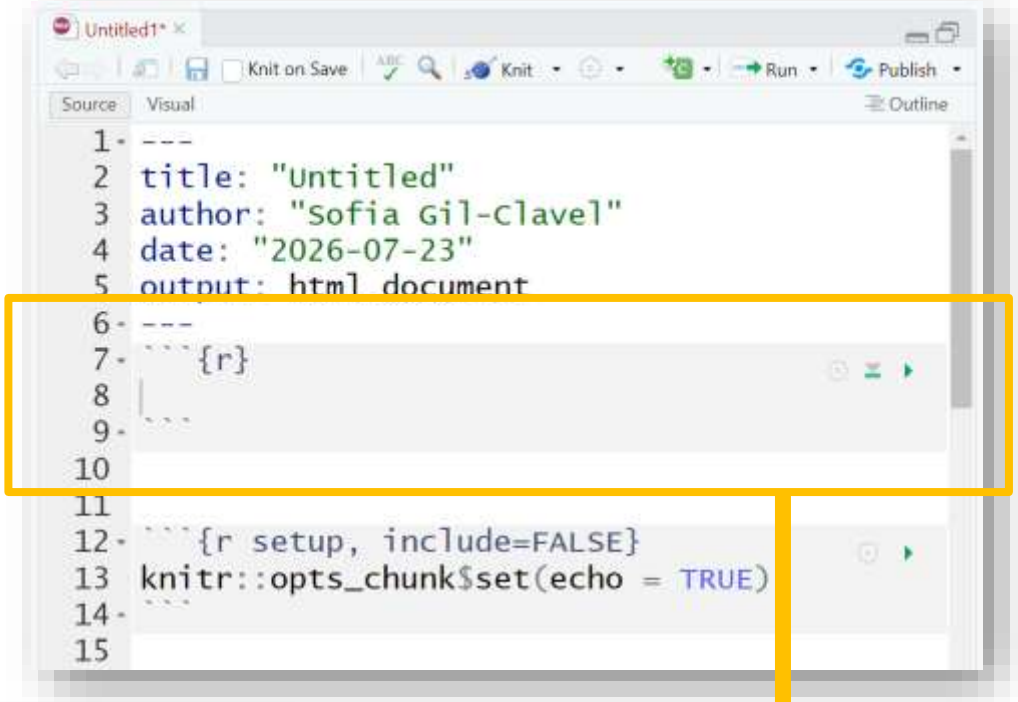


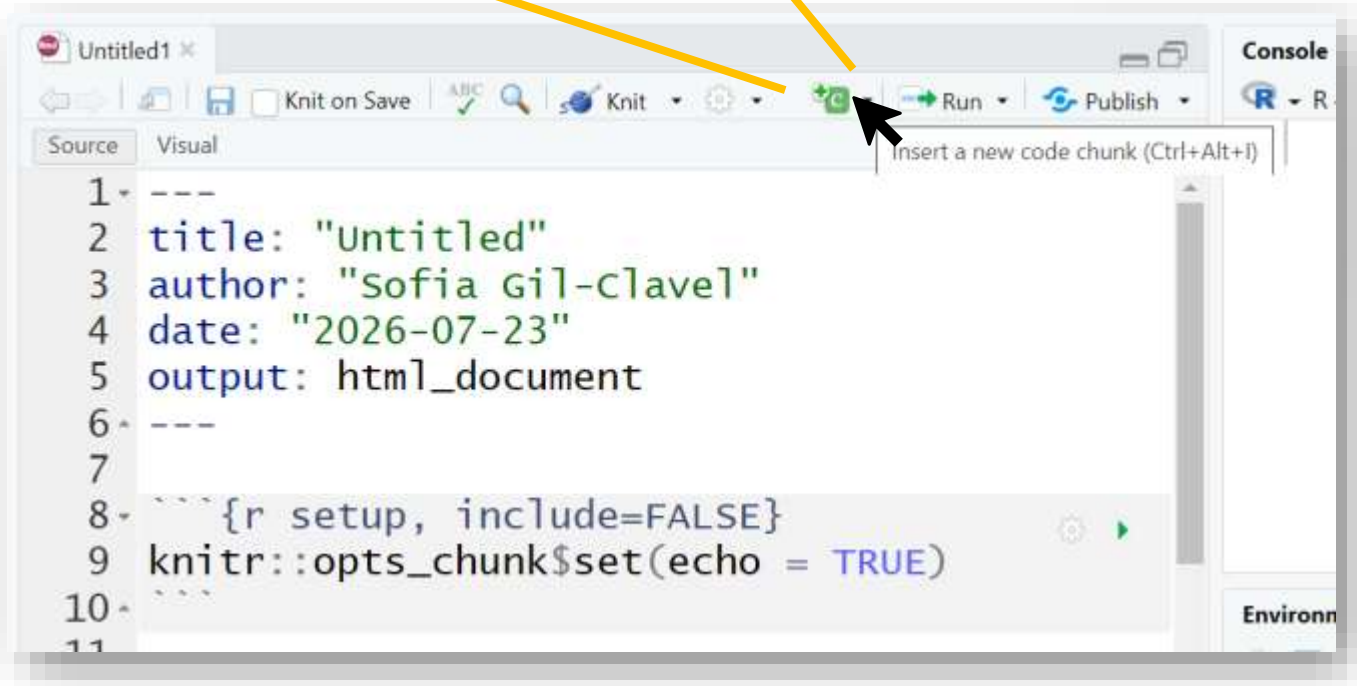


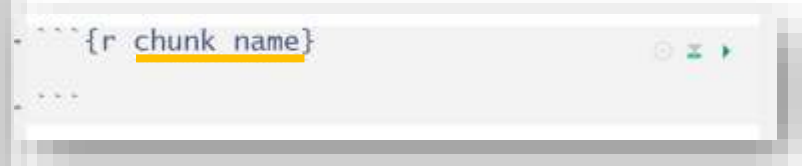
THE CODE GOES IN CHUNKS!

THE CODE GOES IN CHUNKS!





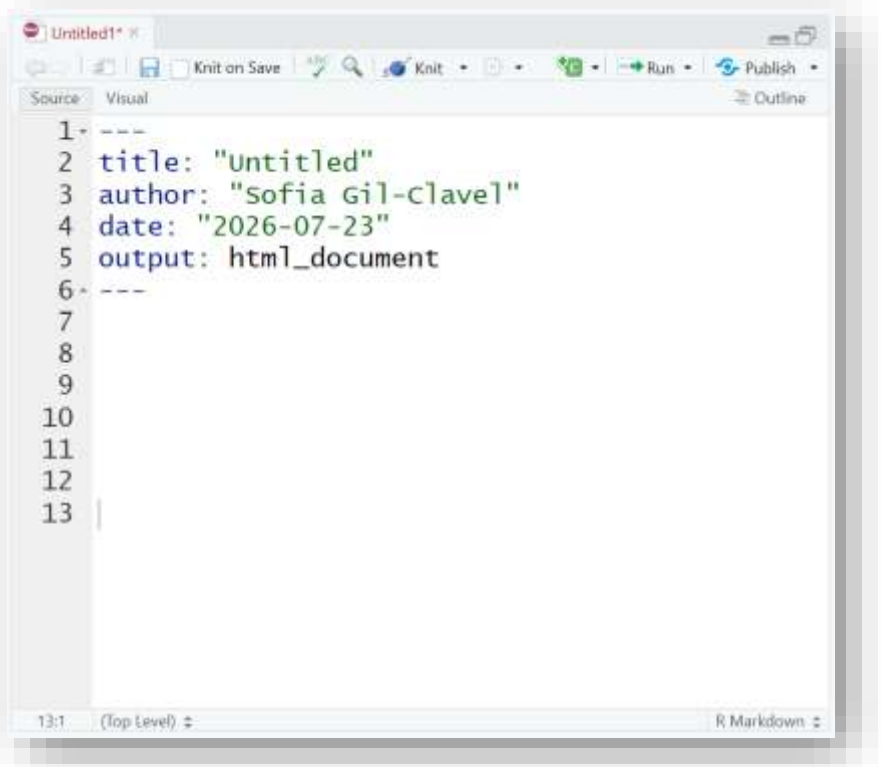




Chunk names must be unique.

EXERCISE 1.1

Delete all the elements in the R-Markdown, except for the header, i.e. lines 1 to 6.



For each of the following steps create a new chunk with a suitable name:

1. Clean the environment.

   Hint: Use the functions rm(list=ls()) and gc().

2. Load the tidyverse package.

3. Turn the “iris” database into a tibble.

4. Obtain the average of Petal.Length by Species.

   Hint: Use the functions group_by() and summarise().

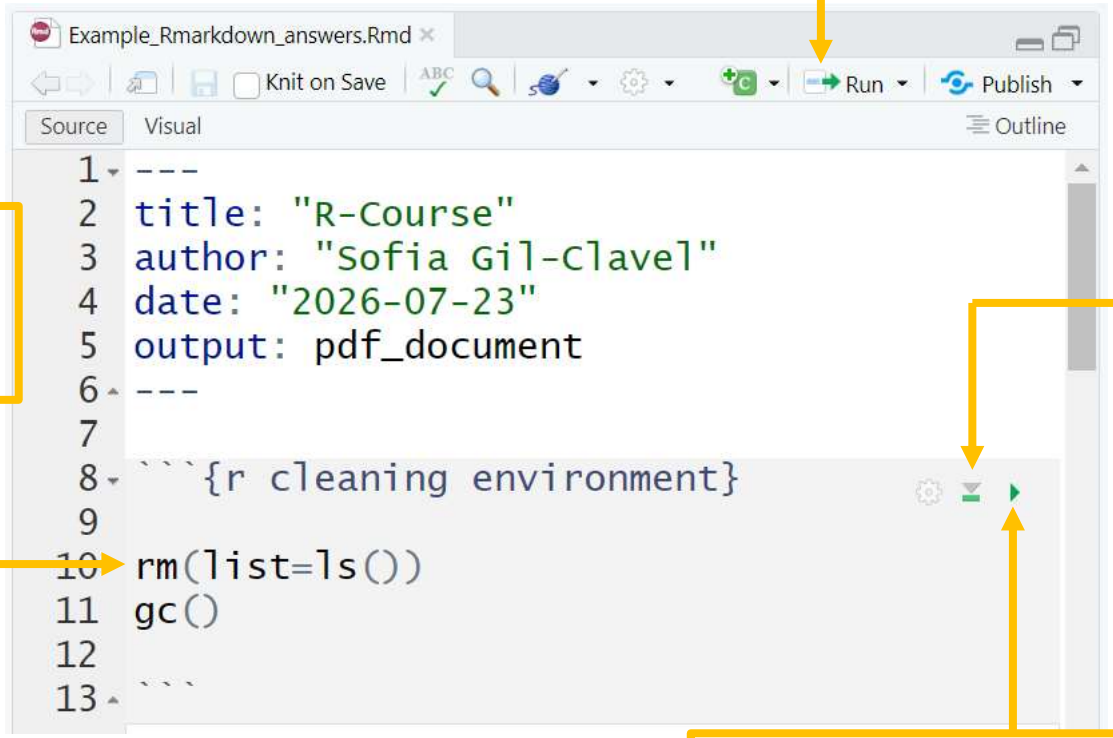
5. Make a bar plot of the results.

RUNNING CODE

Option 2: **Whole chunk.**
Press the "Run" button at the top of the Script panel.

Option 1: **By line.**
Place yourself the line and press the keys "Crtl" + "Enter".

Option 4: **All previous chunks.**
Press the "Downward" play button.

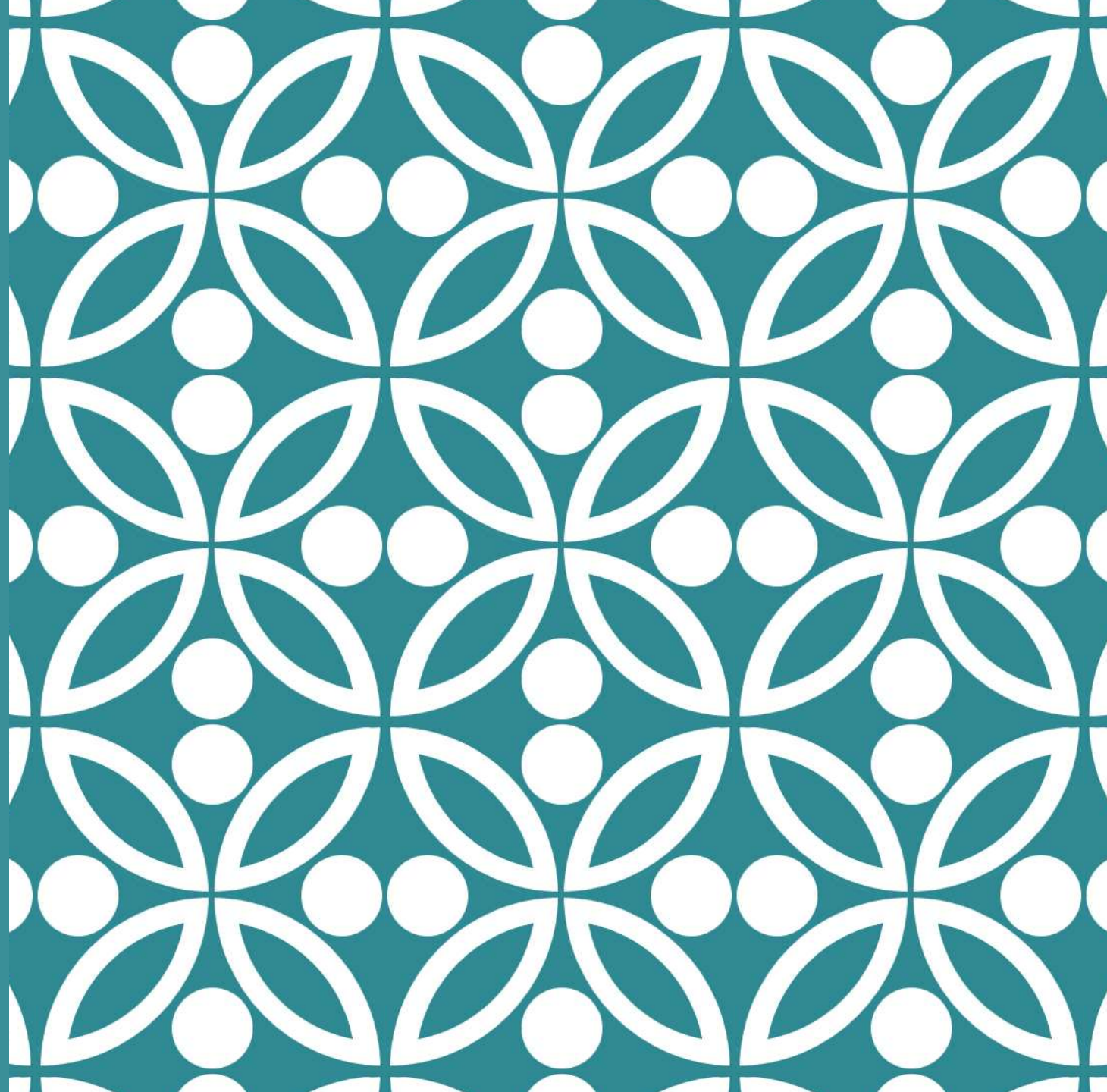


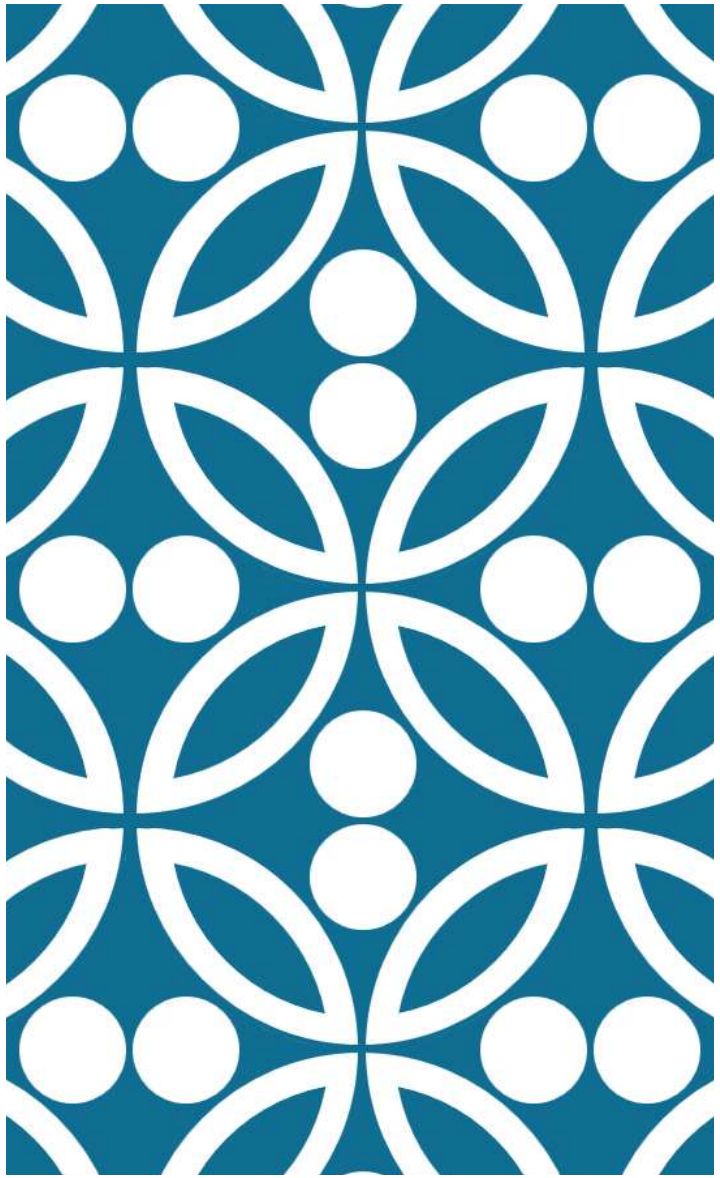
Option 3: **Whole chunk.**
Press the green "play" button at the top of the chunk.

EXERCISE 1.2

1. Go to the final chunk.
2. Run all the previous chunks.
3. Run the final chunk.
4. Render your file!

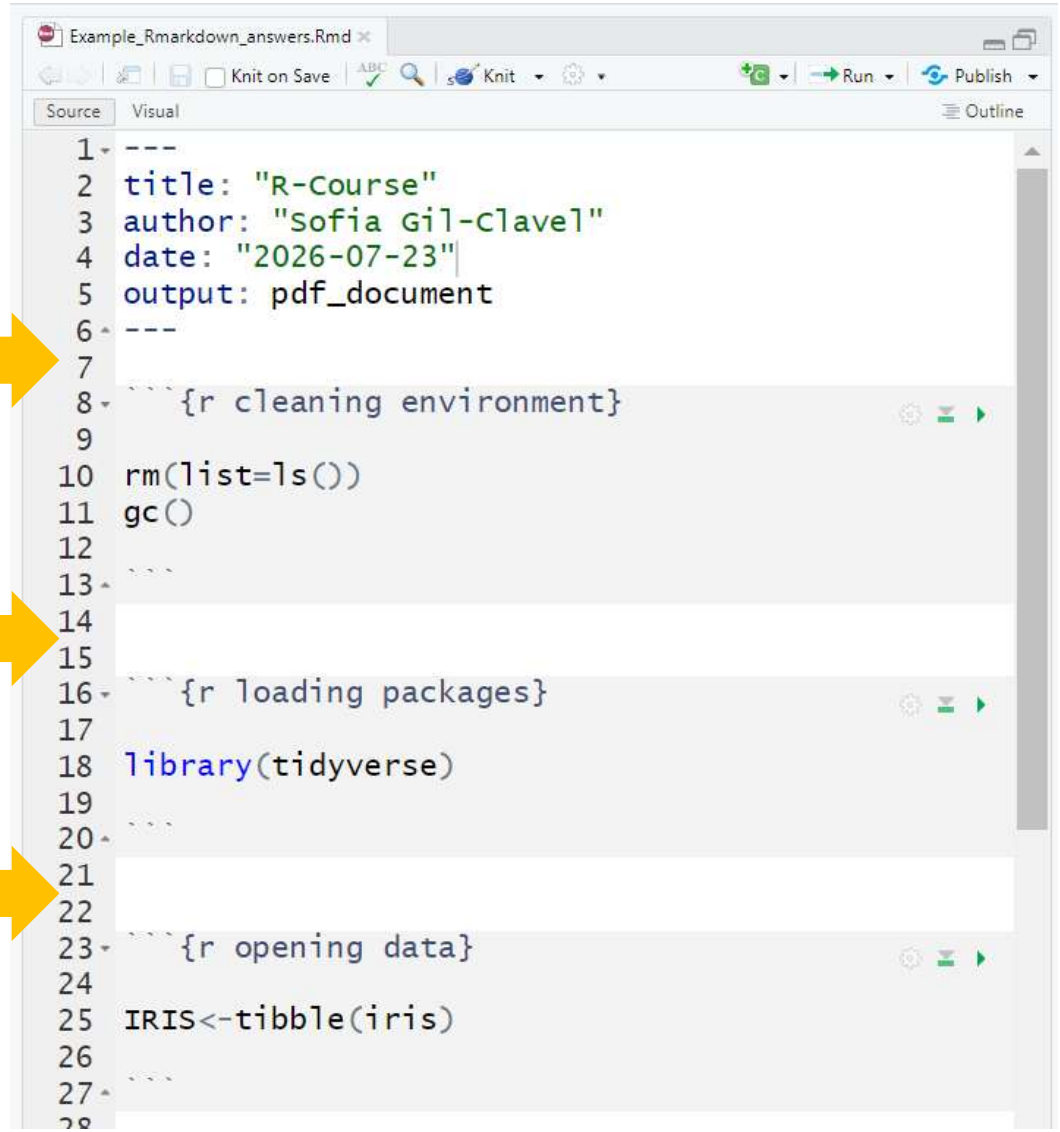
2. ADDING AND FORMATTING TEXT

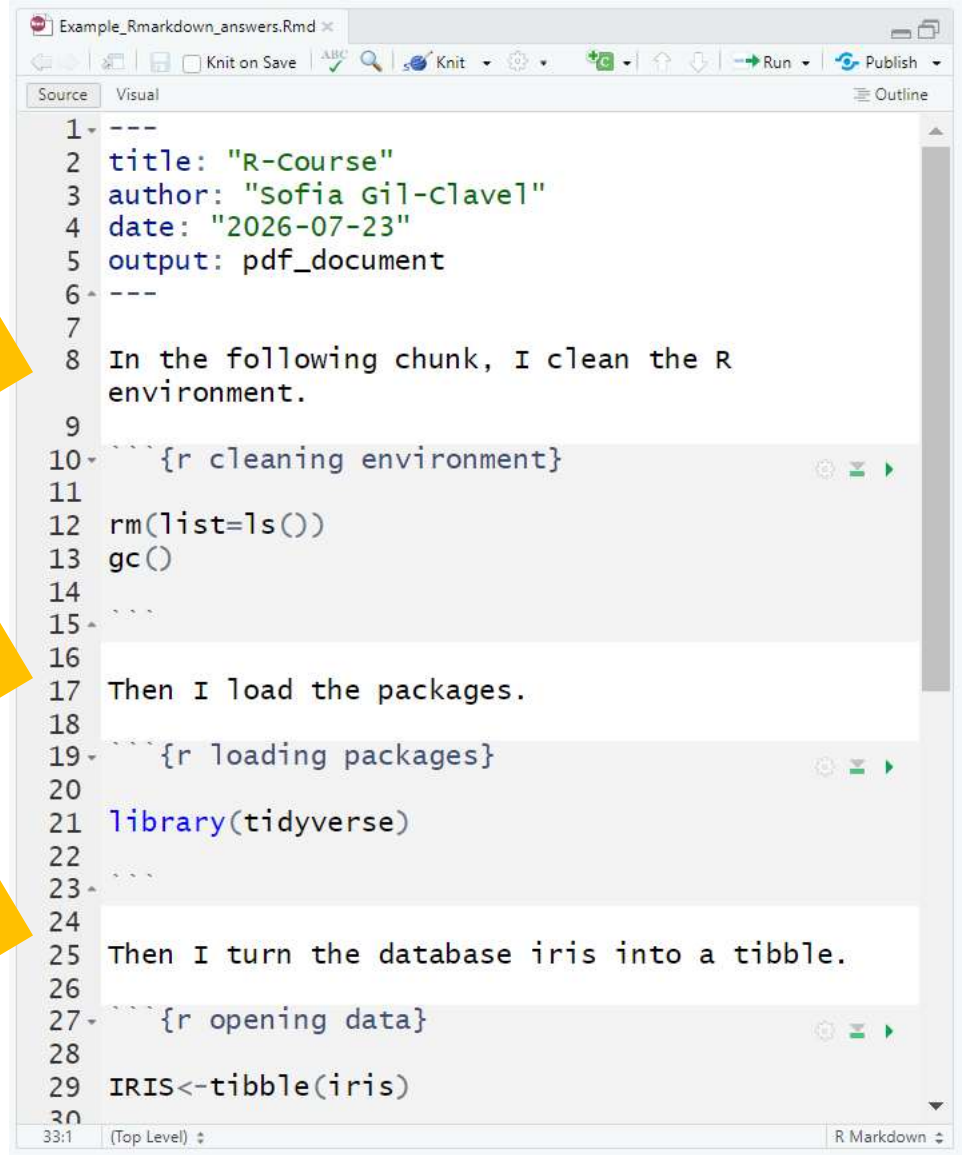




ADDING TEXT

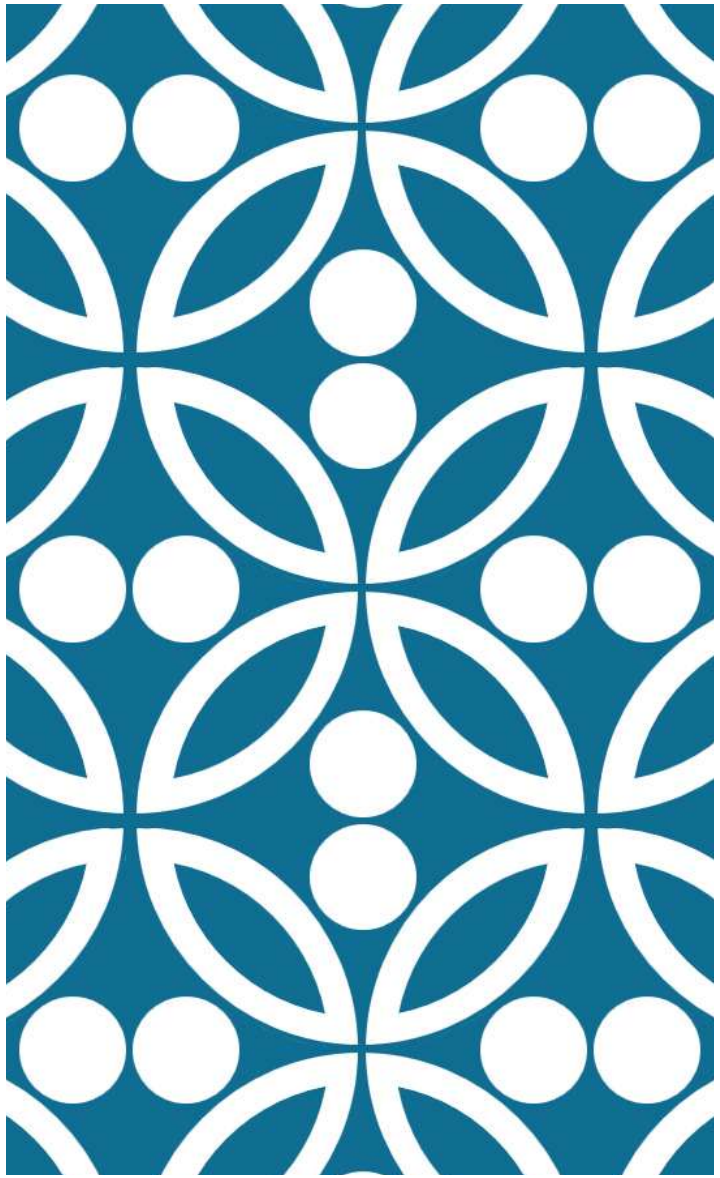
WHITE SECTIONS ARE FOR TEXT





EXERCISE 2.1

1. Write as plain text the description of what you did in each chunk of the R-Markdown.
2. Render the file.

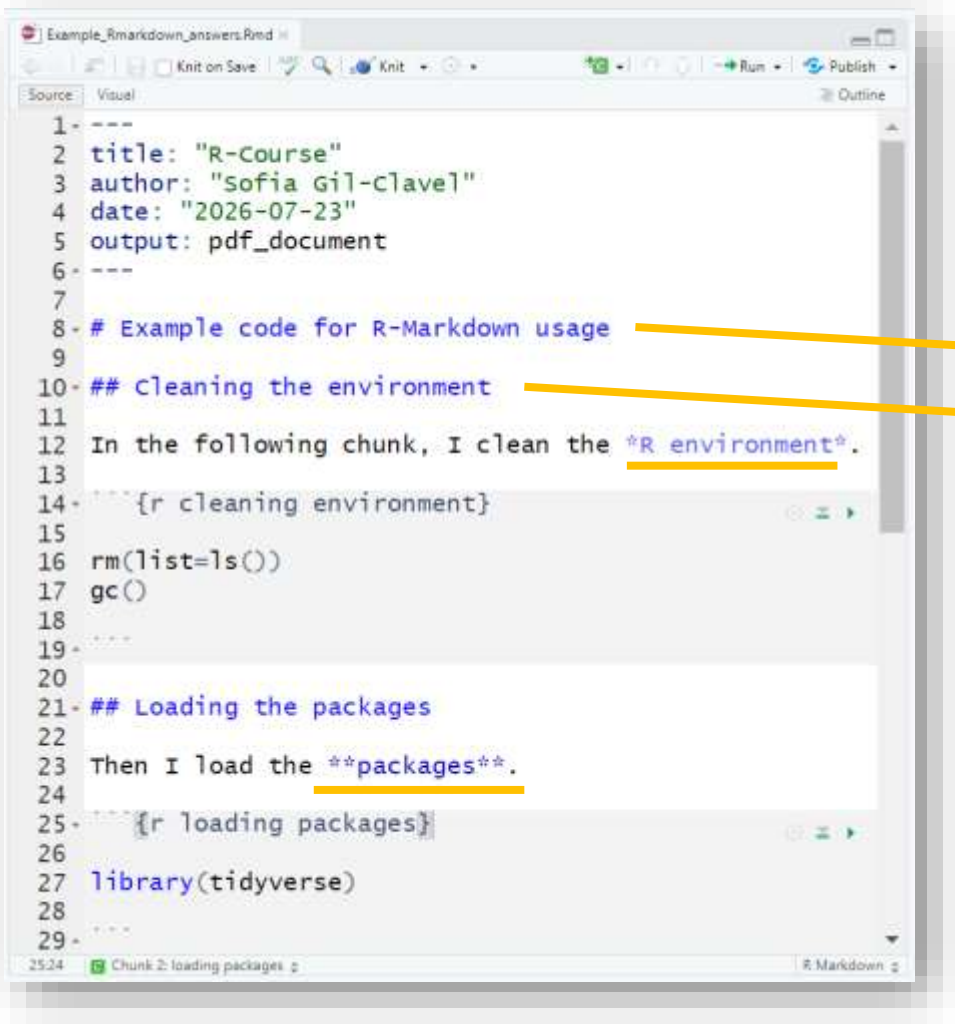


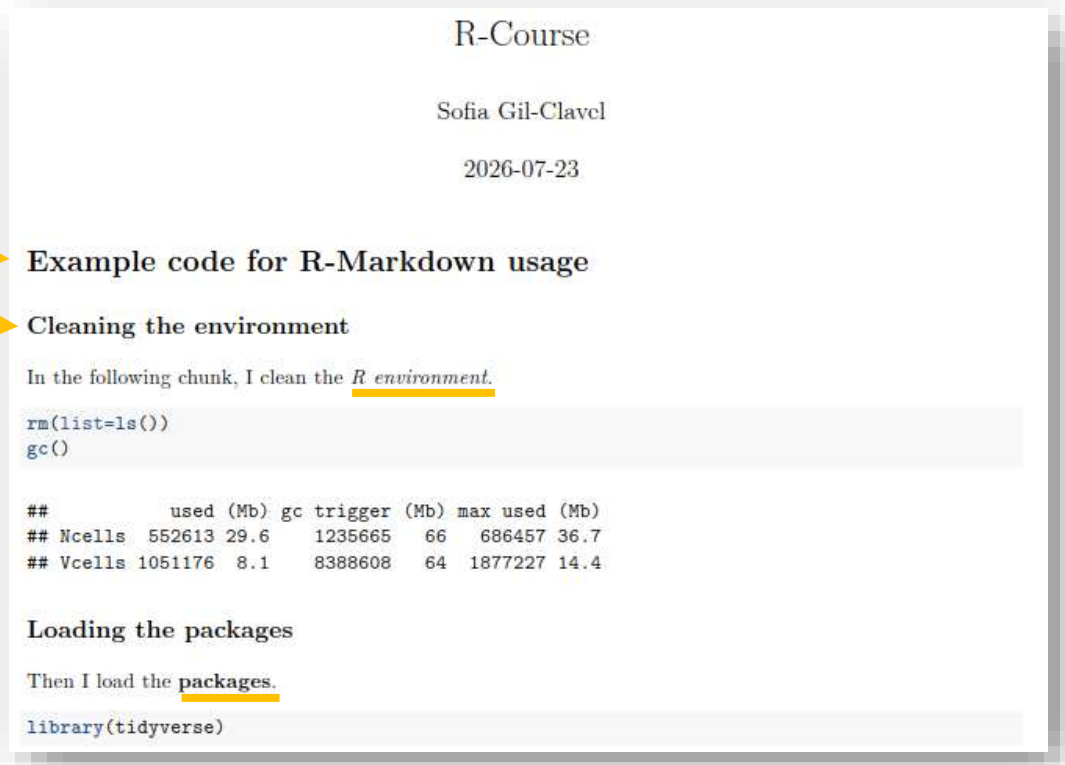
R-MARKDOWN SYNTAX

R-MARKDOWN SYNTAX

You can add headers, use different font styles, create lists, and more!

You just need to use some embedded commands that R-Markdown then will translate in the final document.





R-Course

Sofia Gil-Clavel

2026-07-23

Example code for R-Markdown usage

Cleaning the environment

In the following chunk, I clean the *R environment*.

```
rm(list=ls())
gc()
```

```
##          used (Mb) gc trigger (Mb) max used (Mb)
## Ncells  552613 29.6    1235665   66   686457 36.7
## Vcells 1051176  8.1    8388608   64  1877227 14.4
```

Loading the packages

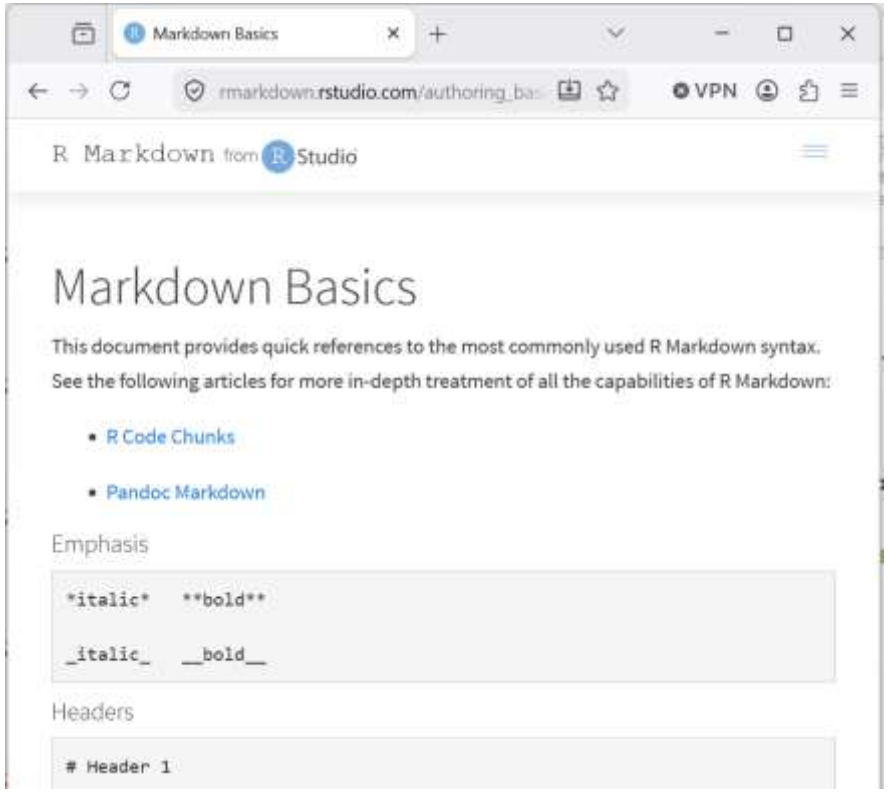
Then I load the **packages**.

```
library(tidyverse)
```

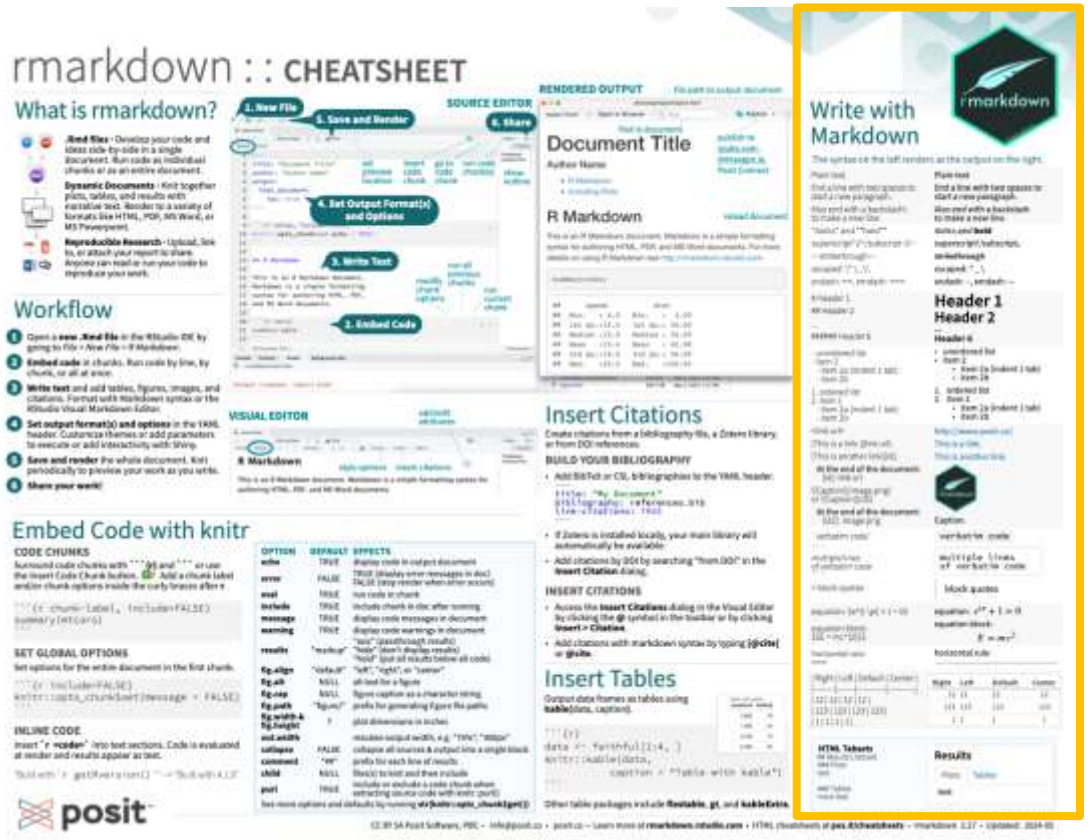
R-MARKDOWN SYNTAX

You can find all the basic commands here:

https://rmarkdown.rstudio.com/authoring_basics.html



You can also find them in the R-Markdown cheat sheet.

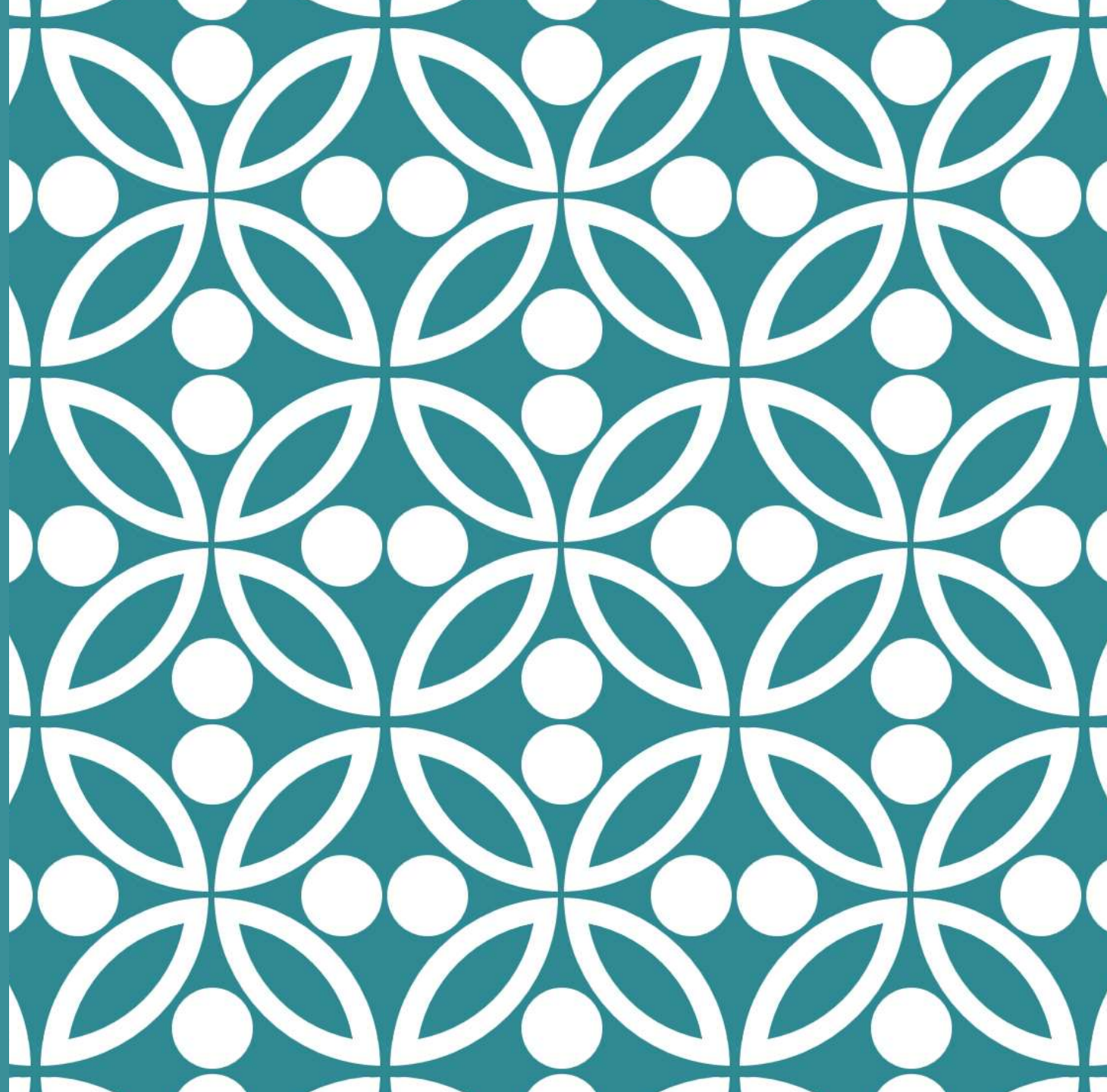


EXERCISE 2.2

Using the R-Markdown syntax from Posit or in the Cheat Sheet add the following elements to your final file:

1. Header
2. Sub-headers that delimit at least the following sections:
   - Preparing the environment
   - Data Handling
   - Data Visualization
3. Three words in italics.
4. Three words in bold.
5. Under the header add the list with the names of the sections in your file.

3. MODIFYING THE SETTINGS



R-Course

Sofia Gil-Clavel

2026-07-23

Example code for R-Markdown usage

Preparing the environment

In the following chunk, I clean the *R environment*.

```
rm(list=ls())
gc()
```

```
##           used (Mb) gc trigger (Mb) max used (Mb)
## Ncells  552618 29.6    1235680   66   686457 36.7
## Vcells 1051244  8.1    8388608   64  1877227 14.4
```

Then I load the **packages**.

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.4.3

## Warning: package 'ggplot2' was built under R version 4.4.3

## Warning: package 'tibble' was built under R version 4.4.3

## Warning: package 'purrr' was built under R version 4.4.3

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.2.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

Sometimes it is not necessary to show all outputs.

Performing Data Handling

For the *data handling*, first, I turn the database iris into a **tibble**.


```
IRIS<-tibble(iris)
```

Finally, I obtain the **mean** petal length by iris species.

```
IRIS2<-IRIS%>%
  group_by(Species)%>%
  summarise(MEAN=mean(Petal.Length))

IRIS2
```

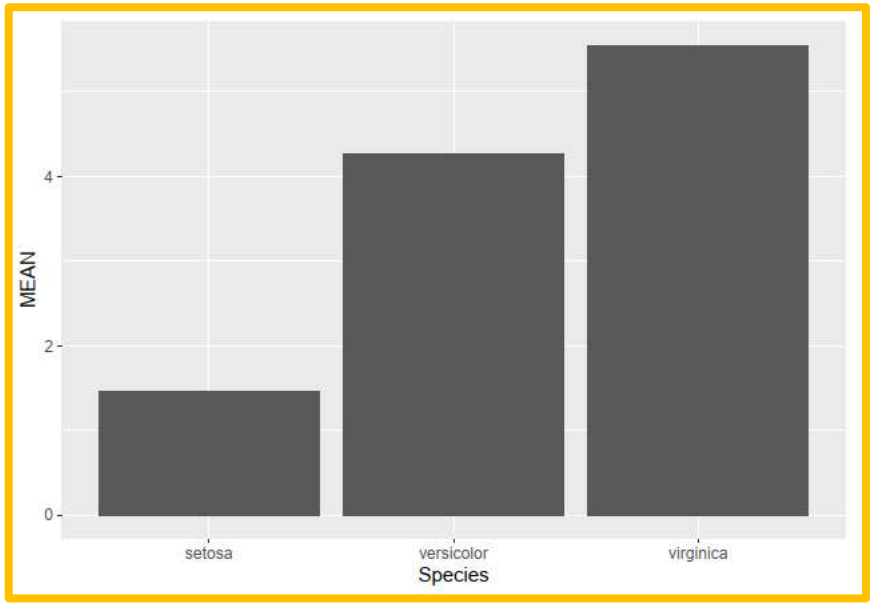
```
## # A tibble: 3 x 2
##   Species     MEAN
##   <fct>      <dbl>
## 1 setosa      1.46
## 2 versicolor  4.26
## 3 virginica   5.55
```

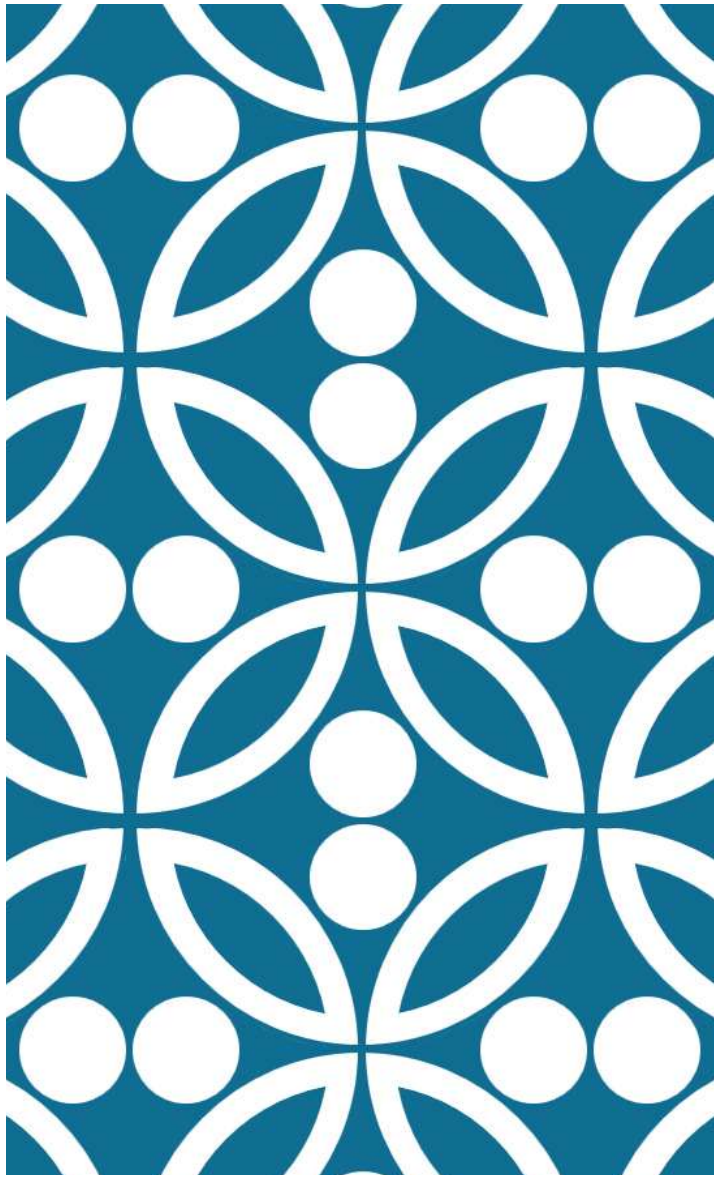
Other times, it is necessary to give better format to the results.

Performing Data Visualization

To visualize the *results*, I plot the values using bar plots.

```
IRIS2%>%
  ggplot(aes(Species, MEAN))+
  geom_col()
```





CHANGING CHUNK PARAMETERS

CHANGING THE SETTINGS

Chunks are like any other function. They have parameters and based on these parameters the format of the code's output is modified.

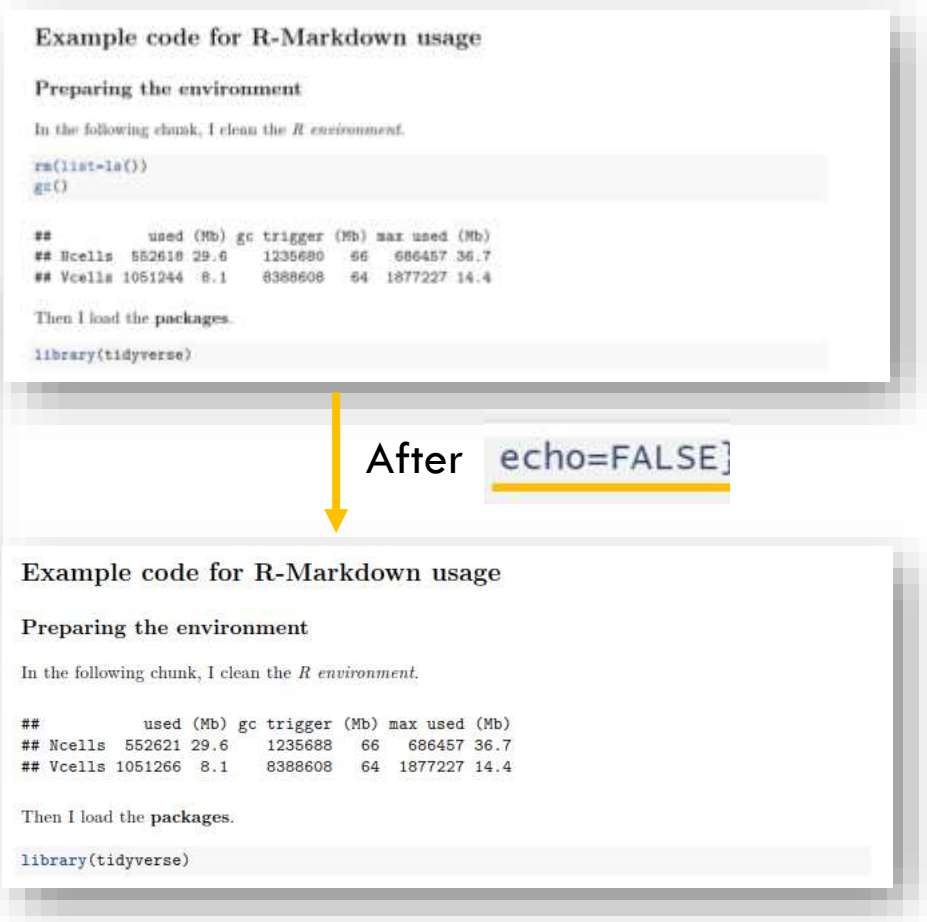
Add a coma after the chunk name and between parameters.

```
```{r cleaning environment, echo=FALSE}

rm(list=ls())
gc()

```
```

This is the parameter "echo" with the assigned value of "FALSE".



CHANGING THE SETTINGS

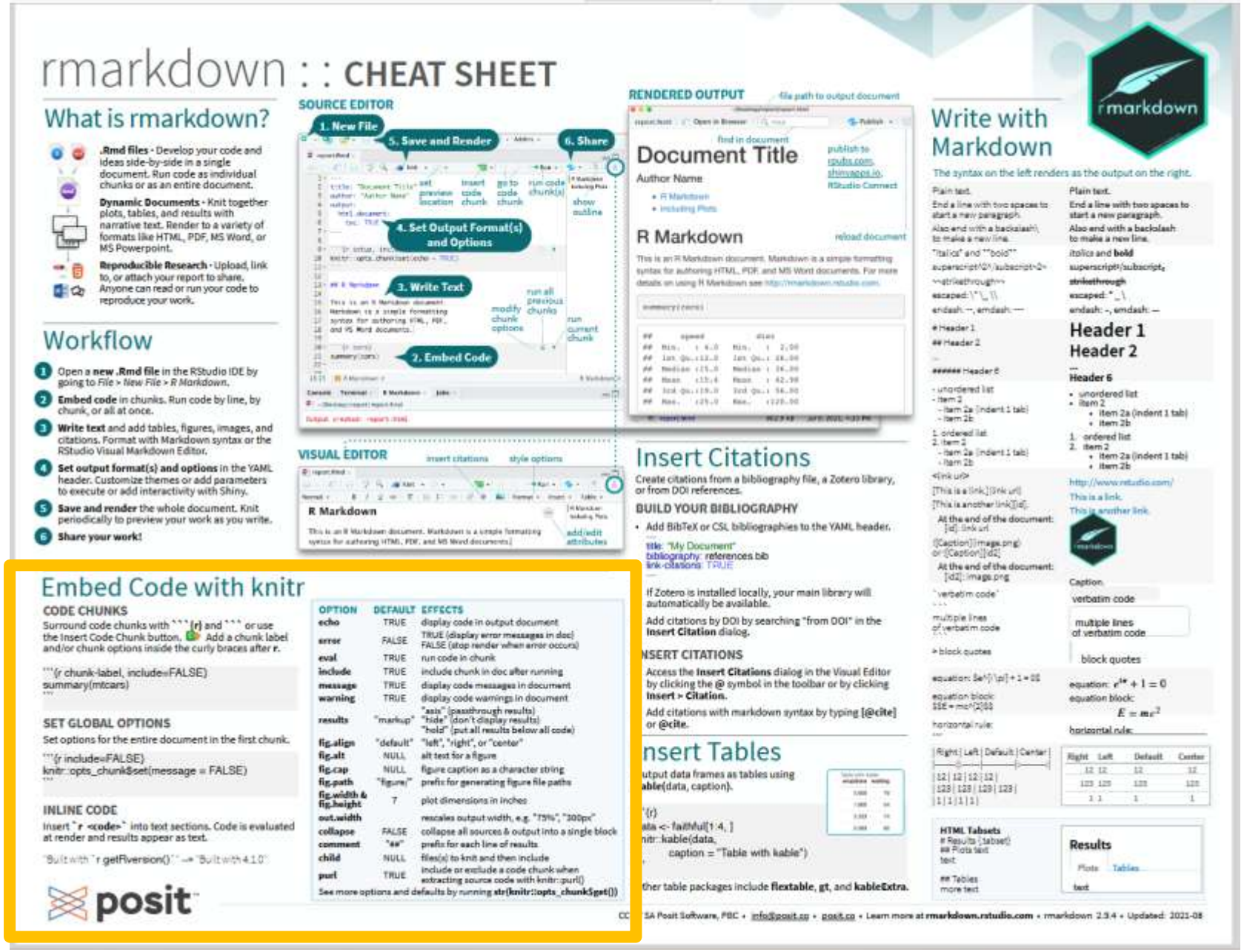
You can find the list of chunk parameters and their description here:

https://pkg.yihui.org/rmarkdown-book/r-code

There are a large number of chunk options in **knitr** documented at https://yihui.name/knitr/options. We list a subset of them below:

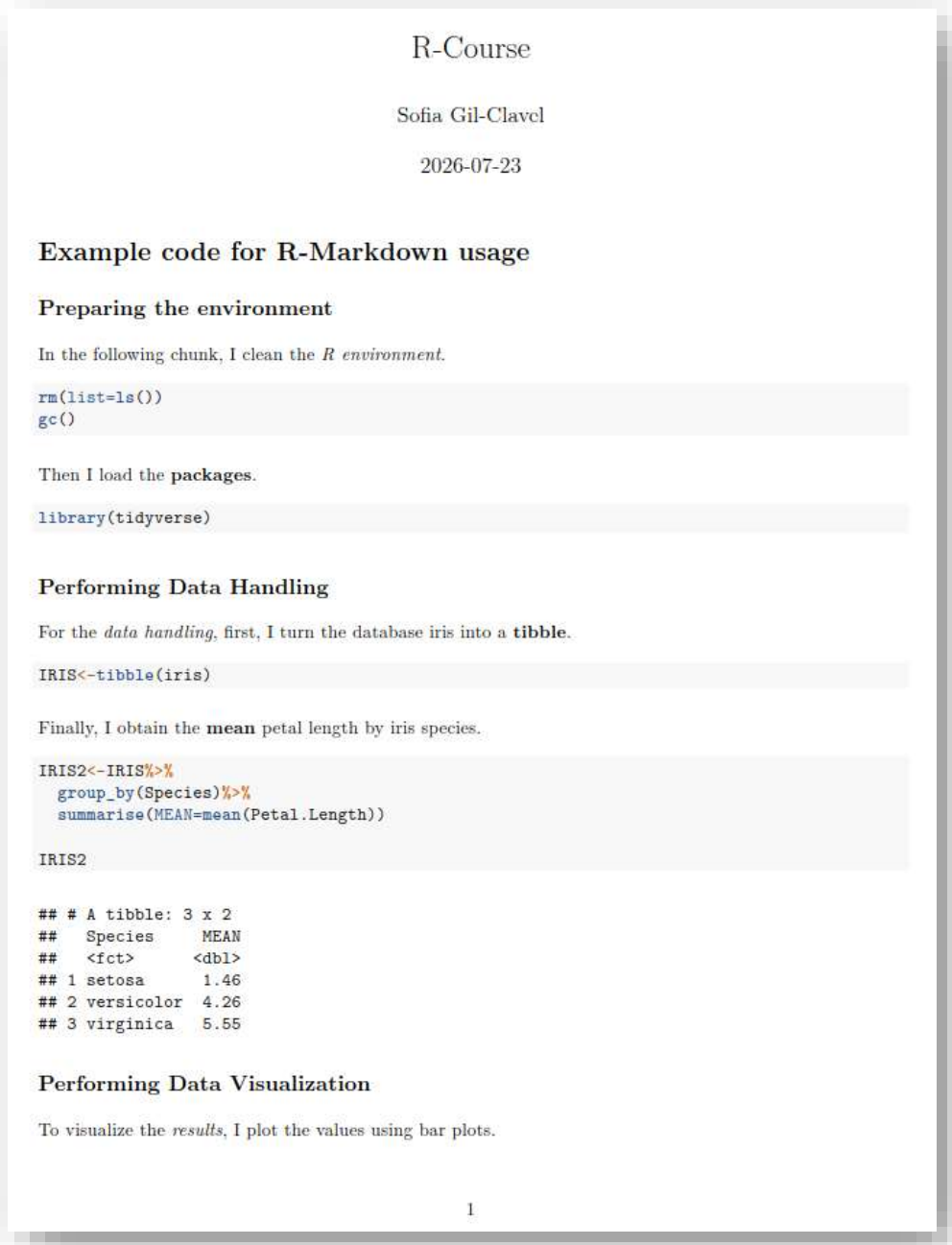
- `eval` : Whether to evaluate a code chunk.
- `echo` : Whether to echo the source code in the output document (someone may not prefer reading your smart source code but only results).
- `results` : When set to `'hide'` , text output will be hidden; when set to `'asis'` , text output is written "as-is," e.g., you can write out raw Markdown text from R code (like `cat('**Markdown** is cool.\n')`). By default, text output will be wrapped in verbatim elements (typically plain code blocks).
- `warning` , `message` , and `error` : Whether to show warnings, messages, and errors in the output document. Note that if you set `error = FALSE` , `rmarkdown::render()` will halt on error in a code chunk, and the error will be displayed in the R console. Similarly, when `warning = FALSE` or `message = FALSE` , these messages will be shown in the R console.
- `include` : Whether to include anything from a code chunk in the output document. When `include = FALSE` , this whole code chunk is excluded in the output, but note that it will still be evaluated if `eval = TRUE` . When you are trying to set `echo = FALSE` , `results = 'hide'` , `warning = FALSE` , and `message = FALSE` , chances are you simply mean a single option `include = FALSE` instead of suppressing different types of text output individually.

CHANGING THE SETTINGS



EXERCISE 3.1

Play around with the different chunk parameters until the first two chunks look like the ones in the image.



R-Course

Sofia Gil-Clavel

2026-07-23

Example code for R-Markdown usage

Preparing the environment

In the following chunk, I clean the *R environment*.

```
rm(list=ls())
gc()
```

Then I load the **packages**.

```
library(tidyverse)
```

Performing Data Handling

For the *data handling*, first, I turn the database iris into a **tibble**.

```
IRIS<-tibble(iris)
```

Finally, I obtain the **mean** petal length by iris species.

```
IRIS2<-IRIS%>%
  group_by(Species)%>%
  summarise(MEAN=mean(Petal.Length))

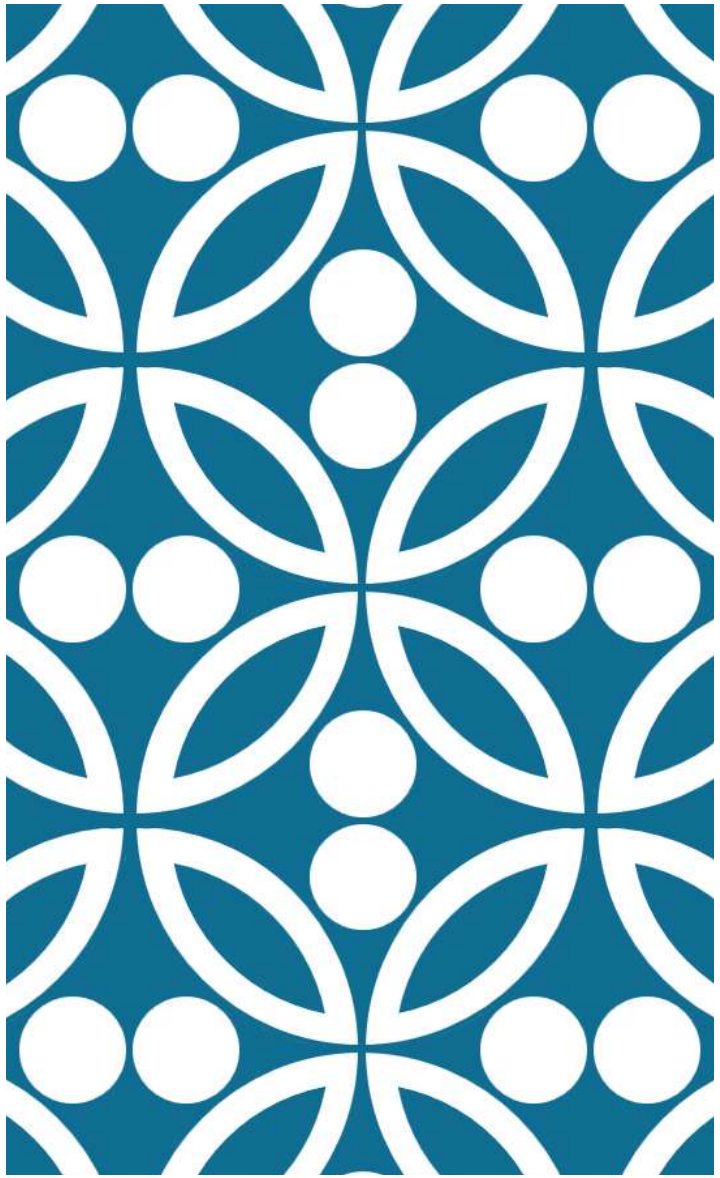
IRIS2
```

```
## # A tibble: 3 x 2
##   Species     MEAN
##   <fct>      <dbl>
## 1 setosa      1.46
## 2 versicolor  4.26
## 3 virginica   5.55
```

Performing Data Visualization

To visualize the *results*, I plot the values using bar plots.

1



CREATING TABLES

CREATING TABLES

```
```{r data handling}

IRIS2<-IRIS%>%
  group_by(Species)%>%
  summarise(MEAN=mean(Petal.Length))

```
```

```
```{r formating as table, echo=FALSE}

knitr::kable(IRIS2)

```
```

To create a table, we will use the function "kable()" from the package "knitr".

You should already have "knitr" installed, as it is used to render the R-Markdown files.

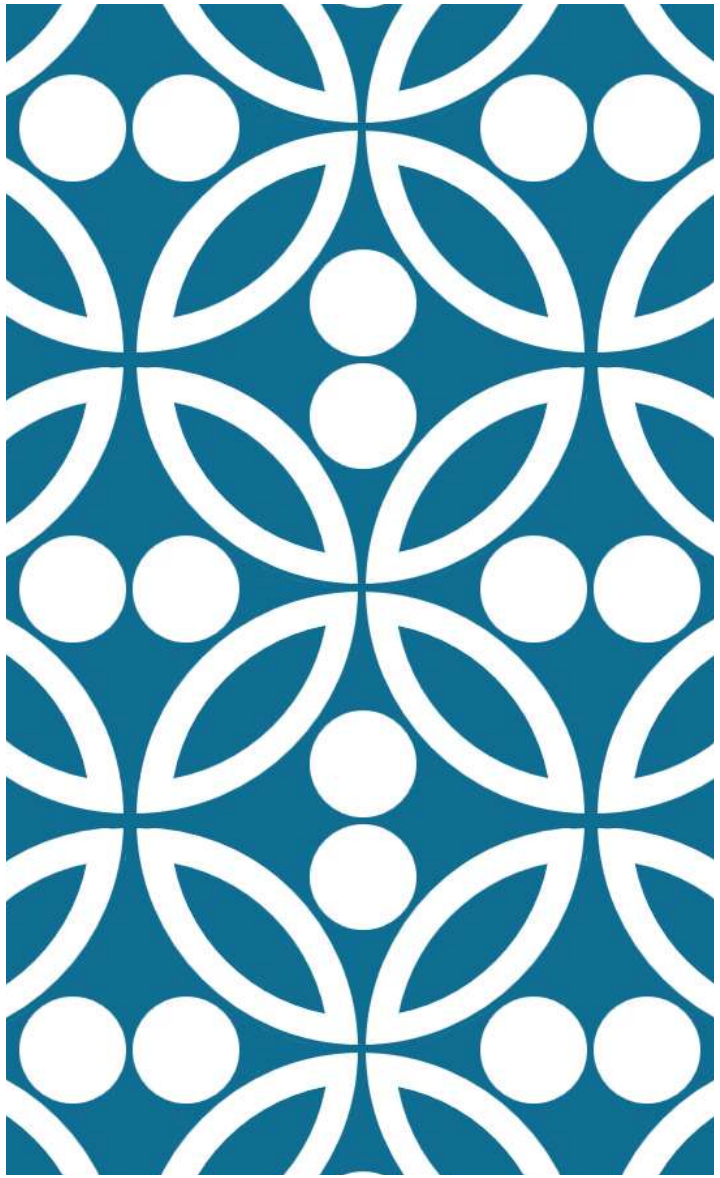
Finally, I obtain the **mean** petal length by iris species.

```
IRIS2<-IRIS%>%
  group_by(Species)%>%
  summarise(MEAN=mean(Petal.Length))
```

Species	MEAN
setosa	1.462
versicolor	4.260
virginica	5.552

EXERCISE 3.2

Display a table with the minimum, maximum, mean, and variance of “Petal.Width” by Species.



FORMATTING GRAPHS

FORMATTING GRAPHS

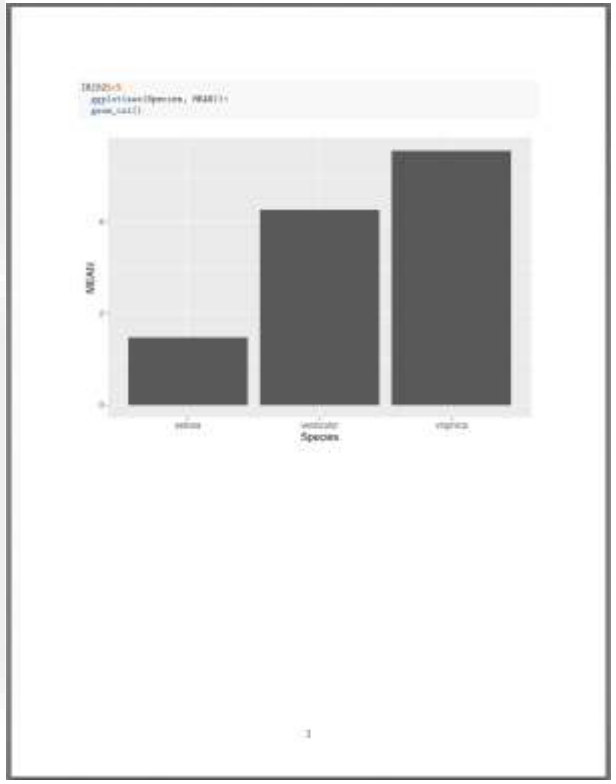
fig.align	"default"	"left", "right", or "center"
fig.alt	NULL	alt text for a figure
fig.cap	NULL	figure caption as a character string
fig.path	"figure/"	prefix for generating figure file paths
fig.width & fig.height	7	plot dimensions in inches

Option 1: At the document level.

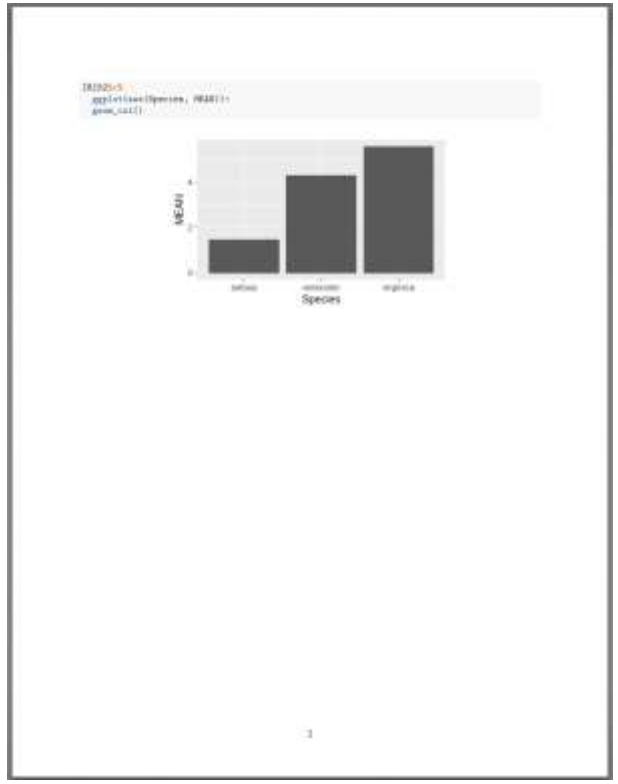
````
14
15 ```{r cleaning environment, results='hide'}
16
17 rm(list=ls())
18 gc()
19
20 ```
````

````
22 ```{r formating the graph, echo=FALSE}
23
24 knitr::opts_chunk$set(
25   fig.align = 'center',
26   fig.height = 2.5, fig.width = 4,
27   fig.cap = NULL,
28   fig.path = "FIGURES/"
29 )
30
31 ```
````

Before

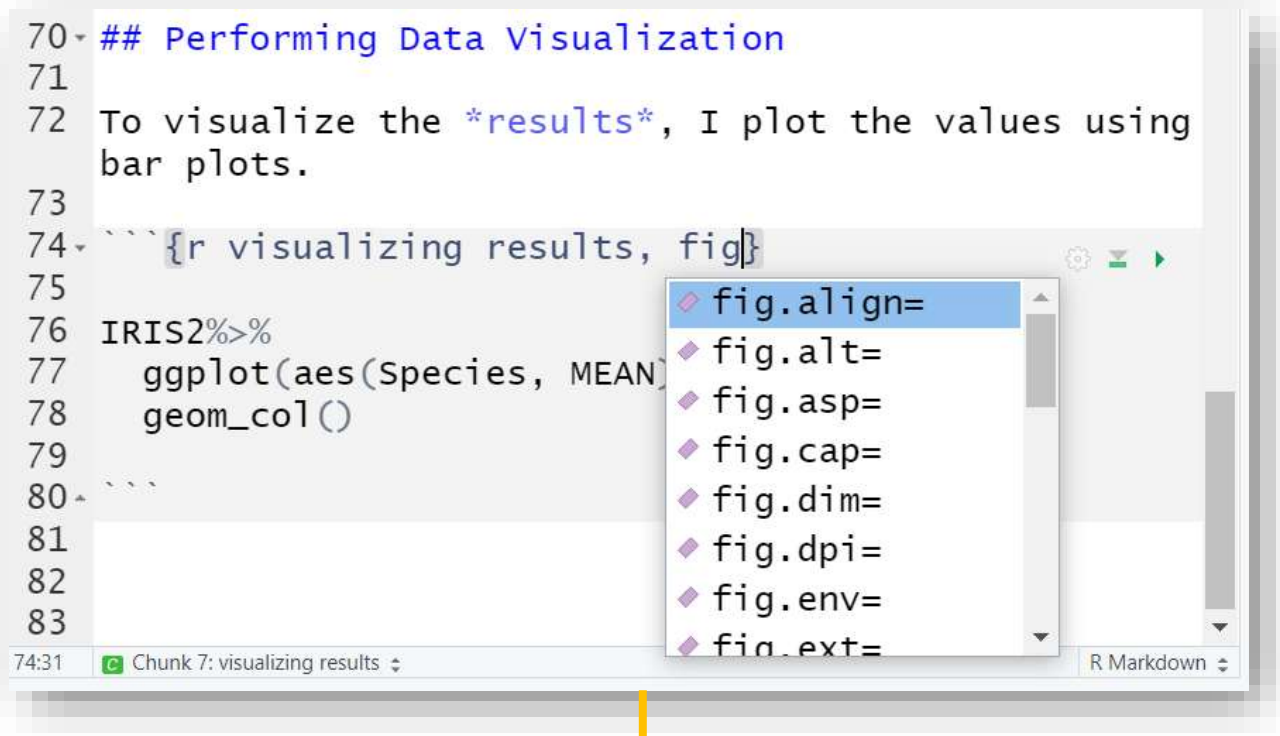


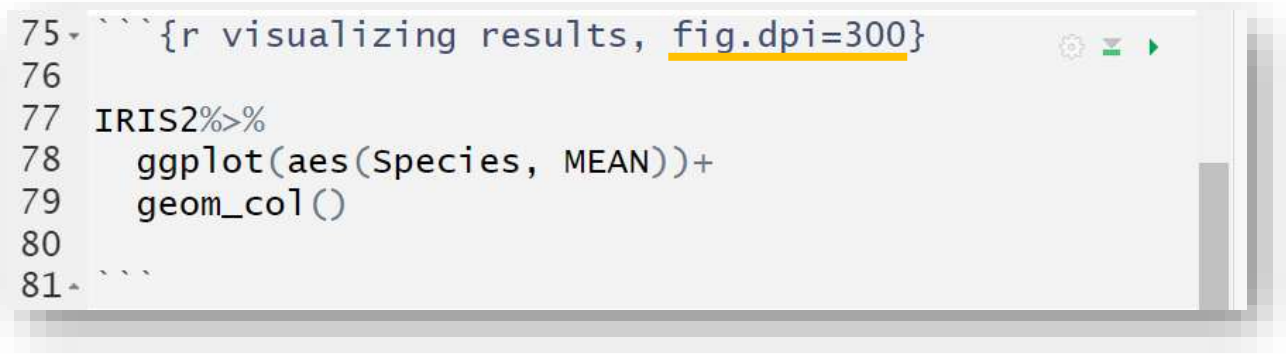
After



FORMATTING GRAPHS

Option 2: At the chunk level, this gives more nuance control, i.e., it gives more parameters to play with.

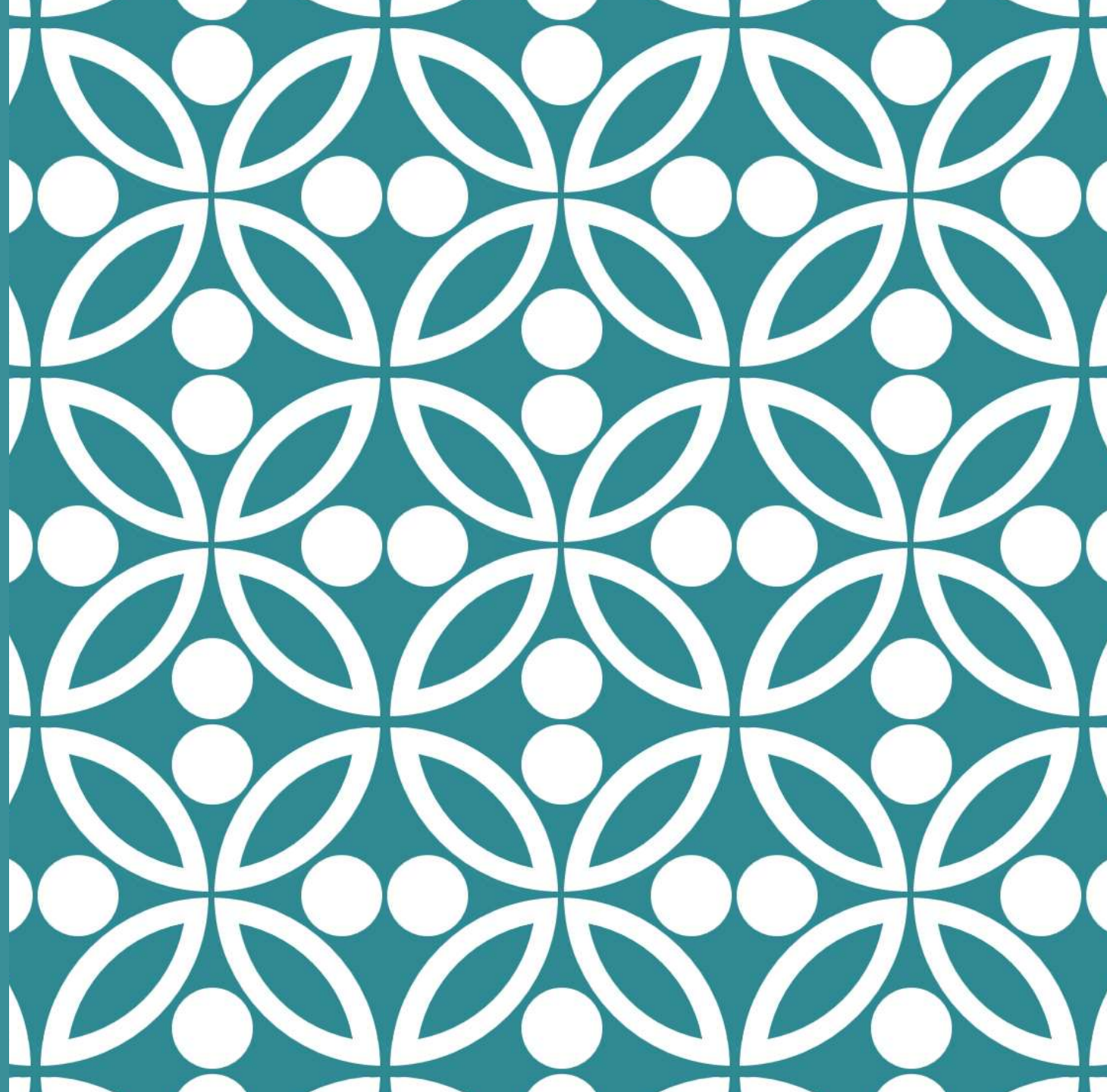




EXERCISE 3.3

Display the box plots of “Petal.Width” by Species.

4. BUILDING ON AXIOMS



THE LOGIC OF MATH

In math **axioms** are the foundational "rules of the game".

An axiom (or postulate) is a statement that is accepted as true without proof.

They serve as the starting points from which all other mathematical truths are logically derived.

The steps to derive knowledge from axioms:

1. **Start with Axioms:** You accept a set of core axioms (and any previous theorems already proven using those axioms).
2. **Apply Rules of Logic:** You use valid logical steps (like *if A implies B, and A is true, then B must be true*).
3. **Reach a Conclusion:** You arrive at a new statement, known as a **theorem**.
4. Once a theorem is proven, it can be added to your toolkit alongside the axioms to help prove even more complex theorems!

MATH LOGIC IN PRACTICE

Using the addition, multiplication, and connecting axioms prove that:

$$(a+b)^2 = a^2 + 2 \cdot a \cdot b + b^2$$

Proof:

$(a+b)^2 = (a+b) \cdot (a+b)$ by exponentiation definition.

$= a \cdot (a+b) + b \cdot (a+b)$ by distributive axiom.

$= a \cdot a + a \cdot b + b \cdot a + b \cdot b$ by distributive axiom.

$= a \cdot a + a \cdot b + a \cdot b + b \cdot b$ by mult. commutative.

$= a \cdot a + 2 \cdot a \cdot b + b \cdot b$ by addition definition.

$= a^2 + 2 \cdot a \cdot b + b^2$ by exponentiation definition.

Addition Axioms

- **Commutative**: a + b = b + a (order does not change the sum).
- **Associative**: (a + b) + c = a + (b + c) (grouping does not change the sum).
- **Identity**: a + 0 = a (adding zero leaves the number unchanged).
- **Inverse**: a + (-a) = 0 (every number has an opposite that yields zero).

Multiplication Axioms

- **Commutative**: a · b = b · a (order does not change the product).
- **Associative**: (a · b) c = a · (b · c) (grouping does not change the product).
- **Identity**: a · 1 = a (multiplying by one leaves the number unchanged).
- **Inverse**: $a \cdot a^{-1} = 1$ for any $a \neq 0$ (every non-zero number has a reciprocal).

Connecting Axiom

- **Distributive**: a · (b + c) = a · b + a · c (multiplication spreads over addition).

MATH LOGIC IN CODING

We can think of functions as axioms and packages as a set of axioms that will serve as foundation to build other code.

The assumption is that the functions in those packages are correct and that if we apply them logically, then the results should also be correct.

The steps to derive knowledge from packages:

1. **Start with a package:** You accept a set of core functions (and any previous functions used to build those functions).
2. **Apply Rules of Logic:** You use valid logical steps (like *if A implies B, and A is true, then B must be true*).
3. **Reach a Conclusion:** You arrive at a new statement, known as a **new function or result.**
4. Once a function is proven, it can be added to your toolkit alongside the packages that helped build even more complex functions and code!

CODING WITH MATH LOGIC

Using the packages base R, pipe, and dplyr prove that the standard deviation of Petal.Length is 1.77.

```
library(dplyr)

iris%>%
  summarise(sd(Petal.Length))
```

```
Console   Terminal ×   Background Jobs ×
R 4.4.2 · ~/
>
>    iris%>%
+      summarise(sd(Petal.Length))
  sd(Petal.Length)
1         1.765298
>
```

Base R

Standard Deviation

Description

This function computes the standard deviation of the values in `x`. If `na.rm` is `TRUE` then missing values are removed before computation proceeds.

Usage

```
sd(x, na.rm = FALSE)
```

Pipe

Basic piping

- `x %>% f` is equivalent to `f(x)`
- `x %>% f(y)` is equivalent to `f(x, y)`
- `x %>% f %>% g %>% h` is equivalent to `h(g(f(x)))`

Here, "equivalent" is not technically exact: evaluation is non-standard, and the left-hand side is evaluated before passed on to the right-hand side expression. However, in most cases this has no practical implication.

Dplyr

dplyr is a grammar of data manipulation, providing a consistent set of verbs that help you solve the most common data manipulation challenges:

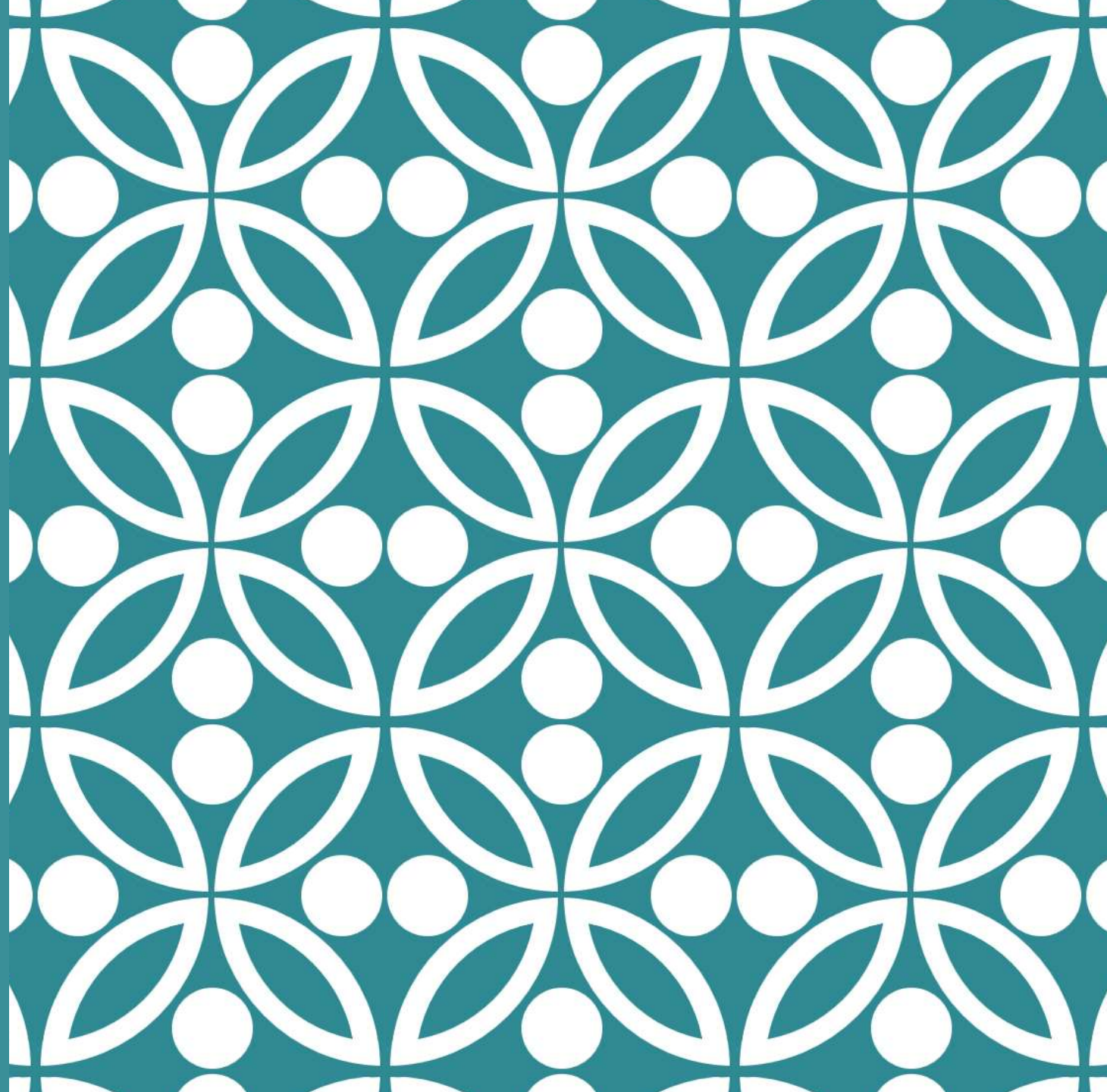
- `mutate()` adds new variables that are functions of existing variables
- `select()` picks variables based on their names.
- `filter()` picks cases based on their values.
- `summarise()` reduces multiple values down to a single summary.
- `arrange()` changes the ordering of the rows.

EXERCISE 4.1

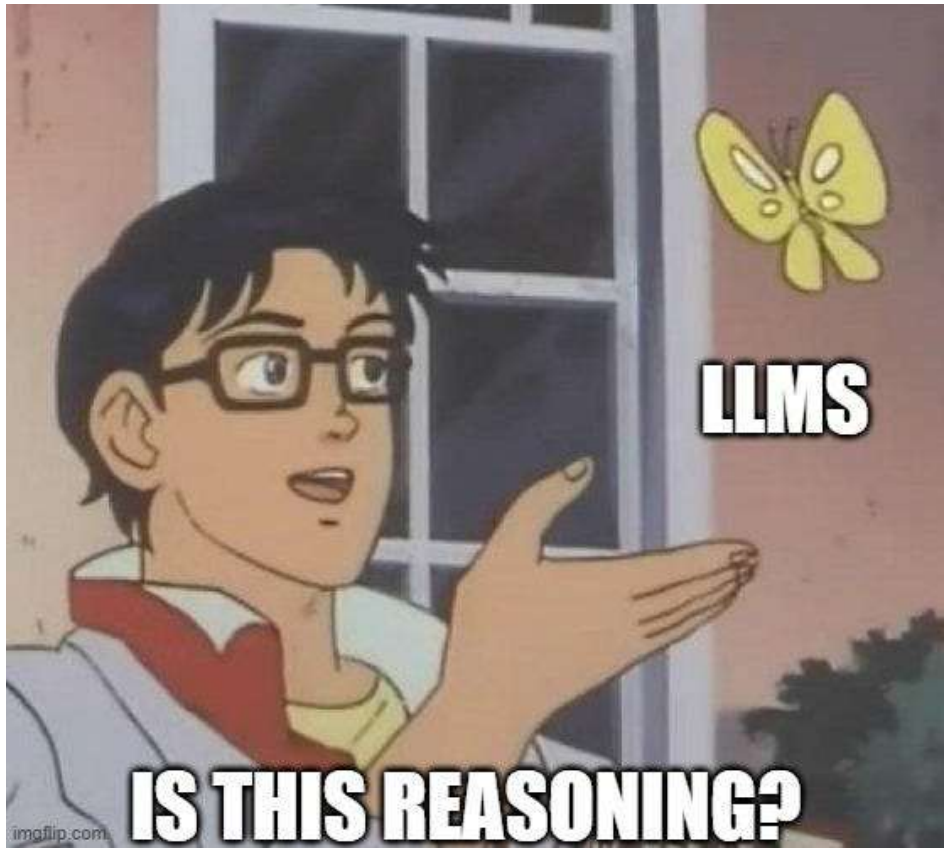
Using just the "stats" package prove that the Virginica species of an Iris flower is on average 4 times bigger than the Setosa species. Explain the logic of each code step.

You should already have the package "stats", as it belongs to Base R.

5. TRACKING STUDENTS' LOGIC

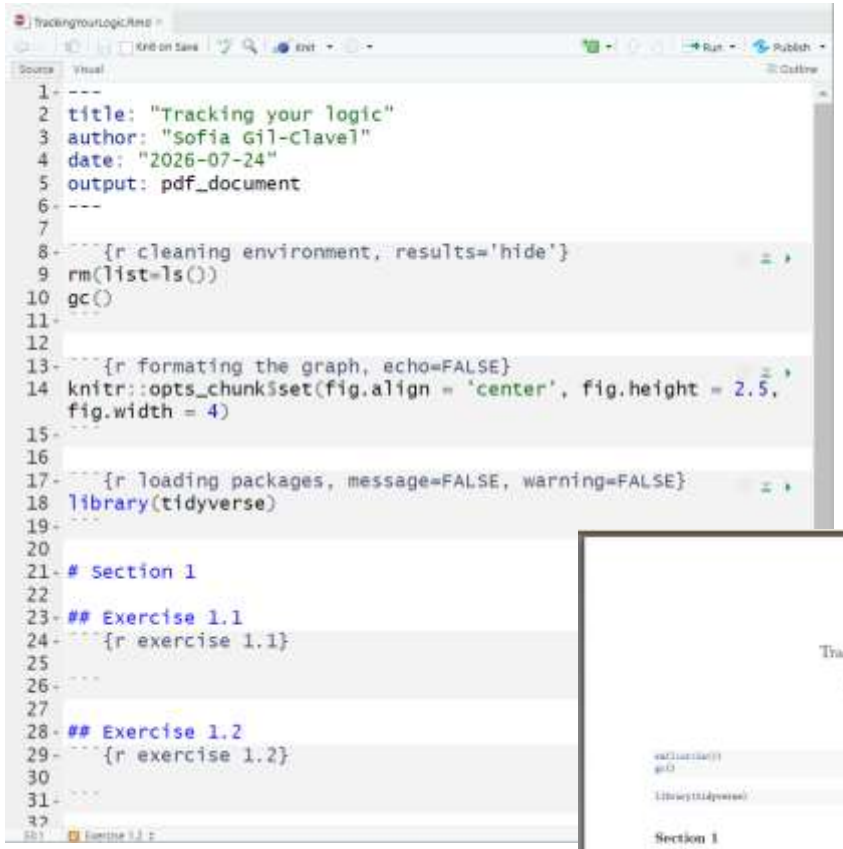


THE NEW CHALLENGE!



R-MARKDOWN ALLOWS TRACKING STUDENTS LOGIC

1. Create an R-Markdown.
2. In the first chunk clean the environment.
3. In the second chunk open the packages that are allowed for the assignment.
4. In the subsequent sections create the chunks where students must write their answers.
5. Request students to submit both the R-Markdown with their answers and the result of rendering the pdf.
6. Remember that the document doesn't render when the code doesn't work.



EXERCISE 5.1

1. Answer the exercises in the R-Markdown “Tracking_Your_Logic.Rmd”.
2. Render the file.

REFERENCES

- Xie, Yihui, J. J. Allaire, and Garrett Grolemund. “R Markdown: The Definitive Guide.” Routledge & CRC Press. Accessed July 21, 2026. https://www.routledge.com/R-Markdown-The-Definitive-Guide/Xie-Allaire-Grolemund/p/book/9781138359338. It can be accessed for free here: https://pkg.yihui.org/rmarkdown-book/

Join us!

https://forms.office.com/e/8Bgd2YsasJ

All about the lab:

https://societal-analytics.nl/

Contact us at:

analytics-lab.fsw@vu.nl.

THANKS!

Dr. Sofia Gil-Clavel

https://sofiag1l.github.io/